Robotics and Autonomous Systems 205 (2026) 105664

Contents lists available at ScienceDirect

Robotics and Autonomous Systems

journal homepage: www.elsevier.com/locate/robot

SGA-TS: A Seeding Genetic Algorithm with TaskShift for task allocation in multi-robot systems

JangHo Seo [a], Joonwoo Lee [a,b,c,d,*]

[a] School of Electronic and Electrical Engineering, Kyungpook National University, Daegu 41566, Republic of Korea
[b] Department of Electrical Engineering, Kyungpook National University, Daegu 41566, Republic of Korea
[c] Department of Smart Mobility Engineering, Kyungpook National University, Daegu 41566, Republic of Korea
[d] School of Aerospace Engineering Sciences, Kyungpook National University, Daegu 41566, Republic of Korea

ARTICLE INFO

Keywords:
Multiple Traveling Salesman Problem (MTSP)
Multi Robot Task Allocation (MRTA)
Seeding Genetic Algorithm (SGA)
Task allocation
SGA-TS
Local search algorithm
Evolutionary algorithm (EA)

ABSTRACT

In deadline-driven Multi-Robot Task Allocation (MRTA), makespan — the completion time of the last robot — is a central objective for evaluating timely mission completion. However, existing approaches lack a path-aware inter-route reassignment operator to reduce flowtime (the sum of all robot completion times) without changing the makespan-first objective function. This paper introduces the Seeding Genetic Algorithm with TaskShift (SGA-TS), a makespan-first MRTA method for grid-based environments in which the objective function exclusively targets makespan minimization, while the TaskShift operator — a deterministic local improvement mechanism integrated into the mutation stage — reduces flowtime by reassigning tasks along robots' computed paths. Experimental results across six grid-based layouts, three robot team sizes, and six baseline algorithms show that SGA-TS achieves the lowest makespan in all 18 tested scenarios and the lowest flowtime in 14 of 18 scenarios. Additional experiments with non-zero service times confirm the consistency of these results.

1. Introduction

The field of robotics has made significant progress in recent years, and robots have become an integral part of many industries. Research on the use of multiple robots has become increasingly necessary for many industries. One of the most active areas of research is the optimization of task distribution among multiple robots, i.e., the so-called Multi-Robot Task Allocation (MRTA) problem. MRTA is crucial for maximizing the efficiency of robots in executing their tasks and enhancing the effectiveness of collaboration between multiple robots. MRTA enables multiple robots to efficiently manage large-scale, complex tasks and is more fault-tolerant than single-robot task execution [1,2]. This approach, which focuses on the optimization and effective distribution of tasks among multiple robots, has been used in a variety of fields, including agriculture [3], natural disaster monitoring [4,5], refinery and industrial inspection [6–8], wildfire monitoring [9], search and rescue missions [10–13], medical facilities [14], and construction [15,16].

MRTA has been studied under diverse objectives, including makespan, flowtime, total travel cost, energy consumption, workload balance, deadline satisfaction, and path feasibility. In deadline-driven ST–SR–TA settings, makespan minimization remains a central concern because it directly measures the completion time of the last robot. This study therefore focuses on makespan-oriented task allocation and sequencing in obstacle-aware grid environments. A variety of approaches have been proposed for this purpose, including meta-heuristic optimization algorithms [17–19], online supervisory control [20], mixed integer programming [21], and hybrid genetic algorithms [22]. However, these single-objective approaches focus exclusively on makespan and do not consider flowtime, i.e., the sum of individual completion times of all robots. Neglecting flowtime can lead to highly unbalanced workloads, where some robots remain idle while others travel excessively, resulting in increased energy consumption and reduced operational efficiency.

To address this limitation, several studies have attempted to optimize both makespan and flowtime simultaneously using multi-objective algorithms [23,24]. While these approaches represent a meaningful advance, they introduce practical challenges. Weighted-sum formulations require careful tuning of weights for each deployment scenario, and the resulting solutions are sensitive to these parameter choices. Pareto-based methods produce a set of non-dominated solutions, but selecting a single operating point from the Pareto front requires additional decision-making criteria. More importantly, in deadline-critical environments, multi-objective formulations may compromise makespan guarantees in pursuit of flowtime improvements, which is undesirable when timely task completion is the primary requirement.

* Corresponding author at: Department of Electrical Engineering, Kyungpook National University, Daegu 41566, Republic of Korea.
E-mail address: jwl@knu.ac.kr (J. Lee).

https://doi.org/10.1016/j.robot.2026.105664
Received 4 June 2025; Received in revised form 10 July 2026; Accepted 26 July 2026
Available online 30 July 2026
0921-8890/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Despite these efforts, a gap remains in the literature: few studies have explicitly investigated a path-aware inter-route local improvement operator that reduces flowtime while retaining a makespan-first objective function. Here, "path-aware" refers to reassignment candidates triggered by the computed cell-level path traversal (i.e., transferring waypoints that a robot physically passes along its planned route to that robot), rather than modifying the fitness function to jointly weight makespan and flowtime. Most existing approaches address the makespan–flowtime trade-off at the fitness-function level, which couples the two objectives and often introduces additional tuning and selection burdens in deadline-oriented settings. A more practical strategy for deadline-oriented applications would be to maintain a makespan-focused objective while incorporating a separate operator that improves flowtime by eliminating path-level inefficiencies during the optimization process.

To efficiently implement MRTA, it is critical to accurately represent the environment map in which robots operate, enabling precise path planning and task assignment. There are four different ways to represent maps: polygonal, free-space maps, grid, and path maps [25]. Among these approaches, the grid representation method is recognized in particular for its outstanding performance and is the most widely used due to its high efficiency [26]. The grid representation technique splits the region into a matrix of rectangular units, where each unit represents an obstacle or an open space. Many MRTA strategies — spanning GA-based methods [6,27], memetic algorithms [7], ACO [28], and other metaheuristics [29,30] — have demonstrated good performance using this grid representation technique. Therefore, this paper proposes an MRTA framework that leverages the grid representation technique for environment map representation.

This study follows the MRTA taxonomy of Gerkey et al. [31] and considers the ST–SR–TA setting: Single-Task robots (ST), where each robot executes at most one task at a time; Single-Robot tasks (SR), where each task is assigned to exactly one robot; and Time-Extended Assignments (TA), where the task list is known in advance and each assignment includes an ordered sequence of tasks. This setting models common offline, deadline-driven MRTA scenarios such as warehouse logistics and facility inspection, and allows us to focus on allocation and sequencing quality under grid-based travel costs.

This study proposes the Seeding GA with TaskShift (SGA-TS) to optimize the MRTA. Unlike existing multi-objective approaches that attempt to optimize makespan and flowtime simultaneously at the fitness-function level, SGA-TS adopts a hierarchical design: the objective function focuses exclusively on makespan minimization, while the TaskShift operator — integrated into the mutation stage — reduces flowtime by detecting and eliminating path-level inefficiencies through task reassignment. This operator-level approach maintains the makespan-first priority required in deadline-oriented applications while achieving overall flowtime reduction as a complementary benefit. We adopt the Jump Point Search (JPS) algorithm [32] as an implementation component for accurate distance estimation in grid environments. The main contributions of this study are as follows:

1. **TaskShift: a path-aware inter-route local improvement operator.** We design TaskShift, a deterministic local improvement operator embedded in the mutation stage of the GA. TaskShift generates reassignment candidates from JPS-computed cell-level traversal paths and applies a bottleneck-first rule to reduce path-level flowtime inefficiencies while maintaining the makespan-first evaluation criterion.
2. **A makespan-first framework with flowtime co-improvement.** SGA-TS combines a Seeding GA — whose initial population includes both random and greedy-based chromosomes — with the TaskShift operator to form a hierarchical optimization framework. The fitness function targets makespan minimization, and the TaskShift mutation is used as a separate local improvement operator for flowtime co-improvement.
3. **Comprehensive empirical validation.** We evaluate SGA-TS across six grid-based layouts with varying obstacle complexity, three robot team sizes (5, 10, and 20), and six baseline algorithms spanning GA-based methods, Ant Colony Optimization, and a multi-objective evolutionary algorithm. Additional experiments with non-zero service times support the consistency of the observed trends under the tested service-time setting.

We begin the manuscript with a background and related work section that introduces the key concepts of GA and the Multiple Traveling Salesman Problem (MTSP) and reviews existing MRTA approaches, which is essential to understanding the proposed SGA-TS methodology. We then proceed to explain "SGA-TS: MRTA Using Seeding GA and TaskShift Algorithm" by detailing the integration of these algorithms for efficient task allocation. The 'Simulation Results and Discussion' section presents the rigorous testing of SGA-TS and discusses the benefits of the algorithm for makespan minimization and flowtime reduction. Finally, the conclusion summarizes the findings, discusses limitations, and suggests directions for future research.

2. Background and related work

2.1. Multiple Traveling Salesman Problem (MTSP)

The MTSP builds upon the classic Traveling Salesman Problem (TSP) by involving multiple salesmen, each of whom must travel through various cities. The goal of these salesmen is to minimize the total travel distance or time for all combined. In the MTSP scenario, every salesman starts from a designated point, tours all the assigned cities, and then returns to their original starting point. The main complexity of MTSP lies in effectively coordinating each salesman's route to ensure both the optimization of individual paths and the enhancement of overall system efficiency. This is challenging due to the interconnectedness of the routes, where optimizing one route can affect the efficiency of other routes. In this study, we focus on a min–max (bottleneck) variant of the MTSP, where the objective is to minimize the maximum tour cost among all salesmen (i.e., makespan), which aligns with deadline-driven MRTA settings.

To address the MTSP, various strategies are implemented. Evolutionary methods like GA [33] are widely used for their effectiveness in solving complex routing problems. Additionally, heuristic and meta-heuristic approaches, which include ACO [34], artificial bee colony [35], and simulated annealing [36], are employed to find near-optimal solutions. Additionally, increasingly, reinforcement learning methods are being applied to solve the MTSP [37,38].

2.2. Genetic Algorithm (GA)

The GA is an optimization technique introduced by Holland [39] that mimics the evolution of living organisms to solve problems. Based on the genetics of living organisms, it borrows many steps from the actual evolutionary process, such as crossover and mutation, with Darwin's concept of survival of the fittest being used as the fundamental principle. GA is a probabilistic search method that models genetics and evolution, enabling it to efficiently find near-optimal solutions and search for global optima. GAs have been widely applied to combinatorial routing problems such as the TSP and MTSP [33,40].

2.2.1. Terminology in genetic algorithm

Key terms used in GA include:

- Chromosome: A collection of genetic information used for problem-solving.
- Gene: An individual element that makes up a chromosome, representing a part of the solution.
- Offspring: New chromosomes generated from the previous generation's chromosomes.
- Fitness: A measure indicating how well a chromosome solves the problem.

2.2.2. Operators in genetic algorithm

GA employs three primary operators:

- Selection: This involves choosing chromosomes based on fitness to create offspring. Methods include roulette wheel selection, tournament selection, and ranking selection.
- Crossover: This is used to combine the genetic information of two parent chromosomes to generate new offspring. This process allows for the exchange of genetic material between chromosomes, introducing diversity within the population and enabling the exploration of new regions in the solution space.
- Mutation: This introduces alterations to individual genes within a chromosome. By altering the solution, this process can explore uncharted areas of the solution space, potentially finding better solutions.

Algorithm 1 Genetic Algorithm for Multi-Robot Task Allocation [41]

```
Initialize the relevant parameters: number of robots and tasks (n), maximum evolution number (maxgen), population size (popsize).
gen ← 1                                   ▷ Current evolution number
Generate the initial population as the parent generation.
while gen ≤ maxgen do
    for i ← 1 to popsize do
        Calculate the fitness of every chromosome of parent generation.
    end for
    gen ← gen + 1
    Select the fittest chromosomes from the current parent generation and offspring to form the new parent generation.
    for each pair of chromosomes in the parent generation do
        Perform reproduction operation to produce two new chromosomes.
        Perform crossover operation to produce two new chromosomes.
        Perform mutation operation to produce two new chromosomes.
        Calculate the fitness of the new chromosomes and insert them into the offspring.
    end for
end while
Output the chromosome with the largest fitness.
```

The algorithm (Algorithm 1), adapted from the work of [41], outlines a genetic algorithm specifically designed for solving the MRTA problem. This algorithm employs the essential genetic algorithm operations — selection, crossover, and mutation — to effectively process MRTA elements and derive an optimal solution.

2.3. Related work on multi-robot task allocation

MRTA has been formally categorized by Gerkey and Matarić [31] along three axes: single-task vs. multi-task robots, single-robot vs. multi-robot tasks, and instantaneous vs. time-extended assignments. The present study falls into the Single-Task, Single-Robot, Time-Extended Assignment (ST–SR–TA) category, in which each robot executes one task at a time and the full task list is known a priori. Comprehensive surveys of MRTA methods and their taxonomies can be found in [42]; more recently, Chakraa et al. [43] reviewed optimization techniques for MRTA, classifying approaches into exact, heuristic, and metaheuristic categories, and Athira et al. [44] provided a systematic literature review covering MRTA research from 2013 to 2024. Below we review three main solution strategies relevant to this work.

Exact and mathematical-programming approaches. Integer programming and constraint programming formulations can provide optimal solutions for small-scale MRTA instances. Ham et al. [21] applied mixed-integer programming to warehouse scenarios, and Lam et al. [45] proposed an exact method for grid-based task allocation. Although these methods can provide optimality guarantees for the formulated instances, they scale poorly to larger instances involving dozens of robots and hundreds of waypoints, which are common in real-world warehouse and logistics applications.

Non-GA metaheuristic and heuristic approaches. To overcome the scalability limitation of exact methods, various metaheuristic and heuristic algorithms have been applied to MRTA. Zheng and Yang [46] used Ant Colony Optimization (ACO) for joint task allocation and processing-sequence scheduling, and Qizilbash et al. [28] extended ACO to a combined task-allocation and path-finding planner. Ma et al. [29] proposed a niching-based Differential Evolution with an explicit application to multi-robot task allocation, and Xin et al. [30] combined adaptive large neighborhood search with the Kuhn–Munkres algorithm for dynamic MRTA. Janati et al. [47] employed a clustering-based decomposition to simplify the allocation problem. Recent work in *Robotics and Autonomous Systems* has further explored distributed and decentralized non-GA techniques for dynamic MRTA settings. Shan et al. [48] proposed a decentralized auction-based method for time-constrained dynamic collective transport, in which robots predict peer task choices and form coalitions through a suggestion-augmented auction process. Bagchi et al. [49] introduced a dynamic and decentralized energy-aware allocation technique that handles on-the-fly job arrivals among heterogeneous mobile robots without prior knowledge of job times or counts. More recently, learning-based approaches have also been explored: Paul et al. [50] proposed a graph reinforcement learning architecture using capsule attention networks (CapAM) for MRTA with task deadlines and robot capacity constraints. Many of these studies primarily optimize makespan and do not incorporate explicit mechanisms for improving secondary metrics such as flowtime.

GA-based and evolutionary approaches. Because the MRTA problem can be modeled as a variant of the Multiple Traveling Salesman Problem (MTSP), several GA-based MTSP solvers have been adopted for MRTA. Király and Abonyi [51] proposed an interpretable multi-chromosome GA (IMC-GA) for the MTSP, and Sedighpour et al. [40] combined a GA with a 2-Opt local search (GA-2OPT) to refine MTSP solutions. Other studies have designed GAs specifically targeting MRTA: Liu et al. [6] developed SMP-GA, a multi-population GA with seeding for centralized task allocation in grid environments; Jose and Pratihar [52] introduced SHC-GA, which uses a hybrid integer/binary chromosome to represent robot–task assignments; Liu et al. [7] applied a memetic algorithm to multi-robot inspection; and Liang et al. [27] proposed an energy-penalty strategy within a GA framework for multi-robot task planning. In the multi-objective domain, Chen et al. [23] and Yang et al. [24] proposed evolutionary algorithms that simultaneously optimize makespan and flowtime using weighted-sum or Pareto-based formulations.

Recent studies further demonstrate that MRTA and closely related multi-robot routing problems are studied under diverse objectives and algorithmic paradigms, rather than a single makespan-only formulation. For example, Wang et al. [53] developed a multi-objective teaching–learning-based optimizer for cooperative weeding robots and spraying drones, where task allocation must balance multiple operational criteria. Liu et al. [22] proposed a recent hybrid GA for MRTA with limited-span constraints, emphasizing the continuing relevance of evolutionary optimization for deadline-oriented allocation. Learning-based approaches have also expanded beyond CapAM: Park et al. [38] proposed ScheduleNet for the min–max mTSP, and Liang et al. [37] proposed SplitNet, a reinforcement learning method for splitting task sequences in the MinMax mTSP. These methods are important recent developments, but they generally learn or optimize allocation/routing policies under their own task models and do not directly target path-triggered flowtime reduction inside a makespan-first GA.

Another recent line of work explicitly couples task assignment with path planning or collision avoidance. Liu et al. [54] formulated

Table 1
Summary of representative MRTA methods.

Method	Approach	Flowtime handling	Limitation
Ham et al. [21]	MIP	No explicit flowtime objective	Limited scalability
Lam et al. [45]	Exact (grid)	No explicit flowtime objective	Limited scalability
Liu et al. [54]	Integrated TAPF	No explicit flowtime objective	Smart manufacturing setting
Qiu et al. [55]	ILP + LR heuristic	No explicit flowtime objective	Warehouse multi-type robots
David & Valencia [56]	Allocation + trajectory planning	No explicit flowtime objective	Dynamic execution planning
Lee et al. [59]	Voronoi roadmap + redistribution	No explicit flowtime objective	Very large-scale; deadlock-driven
Zheng & Yang [46]	ACO	No explicit flowtime objective	Single-objective formulation
Xin et al. [30]	ALNS + Kuhn–Munkres	No explicit flowtime objective	Single-objective formulation
Shan et al. [48]	Auction (suggestion-augmented)	No explicit flowtime objective	Dynamic collective transport
Bagchi et al. [49]	Decentralized energy-aware	No explicit flowtime objective	Energy/dynamic-arrival focused
Wang et al. [53]	Teaching–learning optimizer	Multi-objective formulation	Domain-specific cooperative robots
Paul et al. [50]	Graph RL (CapAM)	No explicit flowtime objective	Requires training; deadline-focused
Park et al. [38]	RL scheduler	No explicit flowtime objective	Min–max mTSP setting
Liang et al. [37]	RL sequence splitting	No explicit flowtime objective	MinMax mTSP setting
IMC-GA [51]	GA (MTSP-oriented)	No explicit flowtime objective	Primarily MTSP formulation
GA-2OPT [40]	GA + 2-Opt (MTSP-oriented)	No explicit flowtime objective	Primarily MTSP formulation
SMP-GA [6]	GA with seeding	No explicit flowtime objective	Makespan-only optimization
SHC-GA [52]	GA, hybrid chromosome	No explicit flowtime objective	Makespan-only optimization
Chen et al. [23]	Multi-obj. EA	Fitness-level weighting	Weight sensitivity
MO-MOEA [24]	Multi-obj. EA	Pareto-based	Operating-point selection

precedence-constrained multi-agent task assignment and path finding for smart manufacturing, jointly considering task precedence and conflict-free path constraints. Qiu et al. [55] modeled integrated task assignment and path planning for multi-type warehouse robots using a state–time–space network and a Lagrangian relaxation heuristic. David and Valencia [56] combined centralized task allocation with a distributed trajectory planner to coordinate task assignment and collision-free trajectories in dynamic environments. Chakraa et al. [57] integrated collision-avoidance considerations into MRTA for inspection missions, and Yang et al. [58] addressed task scheduling with collision-free path planning in multi-region agricultural operations. Most recently, Lee et al. [59] proposed a very large-scale MRTA method that constructs a Generalized Voronoi Diagram roadmap and redistributes robots to reduce inter-robot conflicts and deadlocks while minimizing makespan in challenging environments with narrow passages. We respectfully distinguish our scope from this coupled execution-planning line: SGA-TS addresses the offline ST–SR–TA allocation/sequencing layer, where JPS-derived costs provide obstacle-aware geometric travel information. Synchronized collision-free multi-robot execution, robot-body constraints, and time-indexed occupancy conflicts are downstream motion-planning problems rather than the optimization layer targeted in this manuscript.

Despite the breadth of these studies, a common pattern persists: existing methods either focus exclusively on makespan minimization, treating flowtime as an afterthought, or attempt to address both objectives at the fitness-function level through multi-objective formulations. The latter approach introduces practical challenges, including sensitivity to weight parameters in weighted-sum methods and the difficulty of selecting a single operating point from a Pareto front. To the best of our knowledge, few prior GA-based MRTA studies have explicitly integrated a path-aware inter-route reassignment operator whose transfer candidates are triggered by computed cell-level traversal paths while preserving the makespan-first priority in the objective function. This operator-level distinction is different from established route-sequence improvement moves such as 2-opt, relocate, Or-opt, and cross-exchange, which operate primarily on abstract route sequences [60,61]. This gap motivates the design of SGA-TS, which addresses flowtime reduction through the TaskShift operator embedded in the mutation stage, independently of the makespan-focused fitness function.

Table 1 summarizes the key characteristics of representative methods.

3. Problem definition

Modern environments that have large-scale logistics, such as warehouses, supermarkets, and airports, install multiple robots to optimize operations. However, the complex layouts in these environments and the distances between multiple waypoints present a significant challenge for efficiently assigning tasks to robots. Timeliness is critical, especially when tasks have strict deadlines. It is essential to optimize the makespan of robot tasks to ensure that the tasks are completed within their respective deadlines. An effective way to solve these problems is to abstract the environments to a grid-based representation where optimal paths can be computed to enable multirobot scheduling. In this study, we implement a scheduling algorithm in which each robot begins at a starting point, travels through each waypoint, and returns to the starting point.

3.1. Relationship to the MTSP

The assignment of robot tasks in an environment represented as a grid can be likened to a variant of the MTSP, which is described as follows:

- The **cities** in the MTSP correspond to tasks or specific locations.
- Each **salesman** represents a robot.
- Waypoints (V') represent the task locations that each robot must visit, while depots (D) serve as each robot's start and end points.

By framing the problem with a focus on minimizing makespan in the context of the MTSP, this study ensures that the tasks are completed efficiently within their respective deadlines.

3.2. Calculating the distance between waypoints with the JPS algorithm

To compute pairwise shortest distances for the MmTSP formulation, this study adopts the Jump Point Search (JPS) algorithm [32], an optimized pathfinding method for uniform-cost grid environments. JPS is built upon the A* algorithm but exploits a key property of uniform-cost grids: many distinct paths between two cells share the same cost due to path symmetry. JPS prunes these symmetric paths by identifying *jump points* — cells where the direction of optimal travel must change — and skipping all intermediate nodes. JPS guarantees the same optimal shortest path as A* while substantially reducing computation time in practice by pruning symmetric paths on uniform-cost grids [32].

The JPS-derived values are employed as edge weights c_{ij} in the MmTSP formulation described below.

Table 2
Summary of notation used in the MmTSP formulation.

Symbol	Description
V	Set of all nodes (vertices)
A	Set of edges, $A = \{(i,j) : i,j \in V, i \neq j\}$
V'	Set of waypoint (non-depot) nodes
D	Set of depot nodes (starting/ending points)
n	Number of robots (salesmen), $n = \lvert D \rvert$
i, j	Node indices, $i, j \in V$
k	Depot index, $k \in D$
c_{ij}	Travel cost from node i to node j (computed via JPS)
x_{ijk}	Binary variable; 1 if the salesman from depot k traverses edge (i,j), 0 otherwise
u_i	MTZ auxiliary variable representing the visit order of node $i \in V'$
L	Maximum number of nodes a salesman may visit
K	Minimum number of nodes a salesman must visit
z	Auxiliary variable representing the makespan (linearization of min–max)

3.3. MmTSP formulation for makespan minimization

A complete graph $G(V, A)$ is characterized by a set of vertices (nodes) V and a set of edges $A = \{(i,j) : i,j \in V, i \neq j\}$.

According to Kara et al. [62], the mathematical notation for the fixed-destination Multidepot Multiple Traveling Salesman Problem (MmTSP) is as follows. In this study, the objective function focuses on the minimization of makespan, while flowtime reduction is achieved through the TaskShift operator at the mutation stage (see Section 4.2). The values derived using the JPS algorithm are employed as edge weights. The formula used to minimize the makespan in a fixed-destination MmTSP is as follows:

c_{ij} = the cost from node i to j, which is derived using the JPS algorithm.

$$x_{ijk} = \begin{cases} 1, & \text{if the agent starting from the } k\text{th node has passed the } i \text{ to } j \text{ edge} \\ 0, & \text{otherwise} \end{cases}$$

$x_{iik} = 0, \quad \forall i \in V, \quad k \in D$

V' denotes the set of waypoint (non-depot) nodes, and D denotes the set of depot nodes (starting and ending points), with $V = D \cup V'$. The number of robots (salesmen) is $n = |D|$.

u_i is the MTZ (Miller–Tucker–Zemlin) auxiliary ordering variable for node $i \in V'$, representing the visit-order position of node i in a route:

$$u_i \in \mathbb{Z}, \quad 1 \leq u_i \leq L, \quad i \in V'.$$

L is the maximum number of nodes a salesman may visit, and K is the minimum number of nodes a salesman must visit. The auxiliary variable z represents the makespan to be minimized.

Table 2 summarizes the notation used in the formulation.

Using the notation above, the MmTSP formulation for the fixed-destination case is stated as follows:

$$\min \; z \tag{1}$$

$$\text{s.t.} \quad z \geq \sum_{j \in V'} \left(c_{kj} x_{kjk} + c_{jk} x_{jkk}\right) + \sum_{i \in V'} \sum_{j \in V'} c_{ij} x_{ijk}, \quad k \in D, \tag{2}$$

$$\sum_{j \in V'} x_{kjk} = 1, \quad k \in D, \tag{3}$$

$$\sum_{k \in D} x_{kjk} + \sum_{k \in D} \sum_{i \in V'} x_{ijk} = 1, \quad j \in V', \tag{4}$$

$$x_{kjk} + \sum_{i \in V'} x_{ijk} - x_{jkk} - \sum_{i \in V'} x_{jik} = 0, \quad k \in D, \quad j \in V', \tag{5}$$

$$\sum_{j \in V'} x_{kjk} - \sum_{j \in V'} x_{jkk} = 0, \quad k \in D, \tag{6}$$

$$u_i + (L-2) \sum_{k \in D} x_{kik} - \sum_{k \in D} x_{ikk} \leq L - 1, \quad i \in V', \tag{7}$$

$$u_i + \sum_{k \in D} x_{kik} + (2-K) \sum_{k \in D} x_{ikk} \geq 2, \quad i \in V', \tag{8}$$

$$\sum_{k \in D} x_{kik} + \sum_{k \in D} x_{ikk} \leq 1, \quad i \in V', \tag{9}$$

$$u_i - u_j + L \sum_{k \in D} x_{ijk} + (L-2) \sum_{k \in D} x_{jik} \leq L - 1, \quad i \neq j, \quad i,j \in V', \tag{10}$$

$$x_{ijk} \in \{0,1\}, \quad i,j \in V, \quad k \in D. \tag{11}$$

In this formulation, Eq. (1) defines the objective: minimizing the auxiliary variable z, which represents the makespan. Eq. (2) linearizes the min–max objective by requiring z to be at least as large as the total travel cost of each salesman $k \in D$; at optimality, z equals the maximum cost among all salesmen. Eq. (3) ensures that each depot k sends out exactly one traveler. It sums across all the paths leading out of the depot to various destinations, and this sum must equal 1. Eq. (4) ensures that each destination j is visited exactly once. It considers both direct paths from the depot to j and paths from other destinations i to j. Eq. (5) ensures that for every destination j from depot k, the number of arrivals equals the number of departures. Eq. (6) ensures that the number of departures from each depot k is equal to the number of returns to that depot, maintaining a balance between outgoing and incoming paths. Eq. (9) ensures that each waypoint i is directly connected to at most one depot, either as the first or last stop in a tour. Equations (7), (8), and (10) jointly form the Miller–Tucker–Zemlin (MTZ) subtour-elimination constraints. Eq. (7) imposes an upper bound on the ordering variable u_i, Eq. (8) imposes a corresponding lower bound, and Eq. (10) enforces pairwise ordering: if some salesman traverses edge (i,j) (i.e., $\sum_{k \in D} x_{ijk} = 1$), the constraint reduces to $u_i < u_j$, thereby preventing subtours among waypoint nodes. Because Equation (4) guarantees that each waypoint $i \in V'$ belongs to exactly one tour, u_i is uniquely determined without a robot-specific subscript (i.e., u_{ik} is unnecessary). Eq. (11) restricts all x_{ijk} to binary values.

Formally, let C_k denote the total travel cost of the salesman departing from depot k, as given by the right-hand side of Eq. (2). The *makespan* is defined as the maximum completion cost among all salesmen:

$$C_{\max} = \max_{k \in D} C_k = z^*,$$

where z^* is the optimal value of the objective in Eq. (1). The *flowtime* is defined as the sum of all individual completion costs:

$$F = \sum_{k \in D} C_k.$$

While the MmTSP formulation above minimizes the makespan $C_{\max}$, flowtime F serves as a secondary evaluation metric; its reduction is addressed by the TaskShift operator (Section 4.2).

4. *SGA-TS*: MRTA using seeding GA and TaskShift algorithm

In multirobot systems, optimizing task assignment among multiple robots is crucial for minimizing operational costs while enhancing overall system efficiency and productivity. This optimization allows for better coordination of tasks and more effective utilization of resources, ultimately improving overall performance. Considering these criteria, the MRTA problem is regarded as an optimization challenge. To address this, the SGA-TS approach has been introduced.

Before this technique is applied, the robots' working space is preprocessed by representing it as a grid map. Subsequently, each robot is assigned a waypoint to traverse using SGA-TS. The framework of this process is illustrated in Fig. 1. Section 4.1 describes the GA part of SGA-TS, while Section 4.2 describes the TaskShift algorithm in SGA-TS.

The SGA-TS process consists of several steps for optimizing the makespan with multiple robots, as summarized in Fig. 1 and formalized in Algorithm 3. The starting and ending points of each robot, as well as the coordinates of the waypoints they must visit, are first

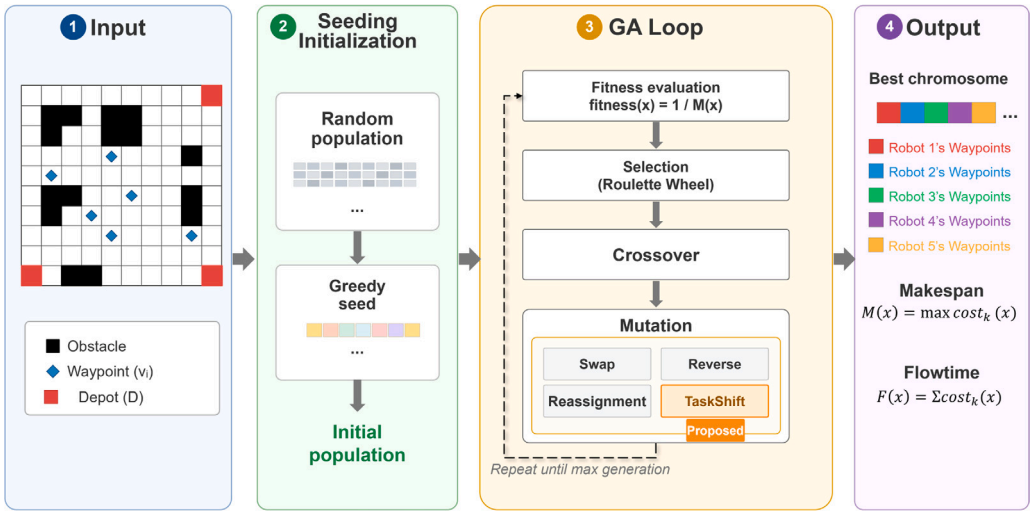


Fig. 1. Overview of the SGA-TS pipeline. The input consists of a grid map with obstacles, robot depot positions (D), and waypoint locations (V'). JPS is used to obtain obstacle-aware segment costs c_{ij} and, when required by TaskShift, the corresponding cell-level traversal paths (Section 3.2). The GA loop (Section 4.1) repeatedly performs selection, crossover, and mutation, where the mutation stage includes Swap, Reverse, Reassignment, and the proposed TaskShift operator (Section 4.2), with the fitness function targeting makespan minimization. The output is the best chromosome, which encodes the robot-to-waypoint assignments with the lowest makespan $M(x)$.

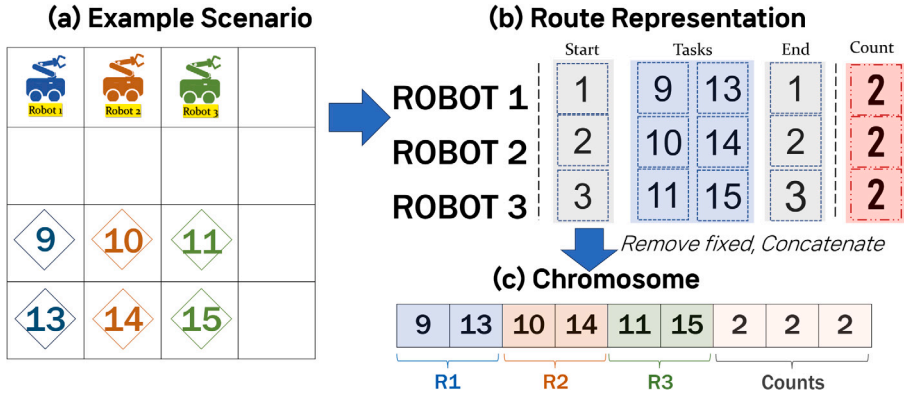


Fig. 2. Chromosome encoding in SGA-TS, illustrated with three robots and six waypoints. (a) Example scenario on a 4×4 grid. Vertices are numbered row-major from the top-left; robot R_k $(k = 1, 2, 3)$ starts and ends at vertex k, and each waypoint is colored by the robot it is assigned to in this example solution. (b) Route representation: each robot's route consists of its fixed start/end vertex (gray, dashed borders) and the vertices of its assigned waypoints, visited in gene order; the last column stores the number of assigned waypoints. (c) Removing the fixed genes and concatenating the per-robot routes yields the chromosome; the last three genes $(2, 2, 2)$ encode the number of waypoints assigned to each robot. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

determined. Next, based on the coordinates of the waypoints and the robots within the grid area, the initial population of chromosomes is randomly generated. At this point, a single chromosome is generated by a greedy nearest-neighbor heuristic that iteratively assigns each robot its closest unvisited waypoint, providing a high-quality seed solution for the MTSP-based formulation (detailed in Algorithm 3, Line 1). The GA process then begins, and it executes activities such as selection, crossover, and mutation. This process iterates until the termination condition is met. During this process, each chromosome is assigned a score by the fitness function, and chromosomes are selected based on these scores. Crossover is then carried out by recombining two selected parent chromosomes. The mutations include Swap, Reverse, and reassignment of the number of assignments. In addition, a deterministic local improvement operator, TaskShift, is applied stochastically to refine solutions (Section 4.2). Through these steps, the SGA-TS seeks to optimize the task assignment and robot routing in a multirobot system to minimize the makespan of task completion and improve flowtime.

4.1. Genetic algorithm of SGA-TS

4.1.1. Chromosomalization of each robot's path

In the scenario presented in Fig. 2(a), robots are situated at vertices 1–3, with six waypoints distributed across vertices 9–11 and 13–15. These robots have the task of efficiently traversing the waypoints. In this example, *Robot 1* (R_1) is assigned the waypoints at vertices 9 and 13, while *Robot 2* (R_2) covers vertices 10 and 14, and *Robot 3* (R_3) covers vertices 11 and 15. As a result, R_1's path includes the sequence of vertices 1–9–13–1. Similarly, R_2's journey includes the sequential vertices 2–10–14–2, and R_3's route includes sequential vertices 3–11–15–3. This routing configuration is depicted in Fig. 2(b).

In Fig. 2(b), the blue boxes represent the vertices that each robot must visit in a sequential order, and the last column indicates the count of mandatory-visit vertices, not including the start and end points. During the process of chromosomal representation, the start and end points (gray boxes with dashed borders) are fixed values, hence there is no need to include them in the chromosome. Removing these fixed genes

Table 3
Notation for fitness evaluation.

Symbol	Description
$R_k(x)$	Route node sequence of robot k: $(v_{k,0}, v_{k,1}, \ldots, v_{k,m_k})$
$\text{Cost}_k(x)$	Total travel cost of robot k in solution x (Eq. (12))
$M(x)$	Makespan: $\max_{k\in D} \text{Cost}_k(x)$ (Eq. (13))
$F(x)$	Flowtime: $\sum_{k\in D} \text{Cost}_k(x)$ (Eq. (14))

and concatenating the per-robot routes yields the chromosome shown in Fig. 2(c): the first six genes list each robot's assigned waypoints in visiting order, and the last three genes store the number of waypoints assigned to each robot.

4.1.2. Fitness function

In our SGA-TS approach, each chromosome is evaluated using a fitness function that reflects the primary objective of minimizing the overall makespan. Let the route assigned to robot k in solution x be the node sequence $R_k(x) = (v_{k,0}, v_{k,1}, \ldots, v_{k,m_k})$, where $v_{k,0}$ and v_{k,m_k} are the depot node and m_k is the number of edges in the route. The total travel cost is

$$\text{Cost}_k(x) = \sum_{p=0}^{m_k-1} c_{v_{k,p}, v_{k,p+1}}, \tag{12}$$

where $c_{v_{k,p}, v_{k,p+1}}$ is the JPS-derived shortest-path cost between consecutive waypoints, consistent with the edge weight c_{ij} defined in Section 3.3.

The makespan $M(x)$ of a solution x is then defined as the maximum travel cost among all robots:

$$M(x) = \max_{k\in D} \text{Cost}_k(x), \tag{13}$$

where $n = |D|$ is the number of robots. In addition, the flowtime $F(x)$ is defined as the sum of individual travel costs of all robots:

$$F(x) = \sum_{k\in D} \text{Cost}_k(x). \tag{14}$$

These quantities correspond to the MmTSP notation in Section 3.3: $\text{Cost}_k(x)$ to C_k, $M(x)$ to $C_{\max}$, and $F(x)$ to F, evaluated for a specific chromosome x.

Here, $\text{Cost}_k(x)$ represents the travel distance only; Section 5.3 extends the evaluation to include non-zero service times at each waypoint. Although the objective function of SGA-TS targets makespan minimization, flowtime serves as a secondary evaluation metric that captures overall workload efficiency. The TaskShift operator (Section 4.2) reduces flowtime at the operator level without altering the makespan-focused fitness function. Table 3 summarizes the evaluation-related notation introduced above.

To minimize the makespan, the fitness function is defined as the reciprocal of $M(x)$:

$$\text{fitness}(x) = \frac{1}{M(x)}. \tag{15}$$

Thus, a lower makespan results in a higher fitness value, ensuring that chromosomes representing more efficient task allocations are favored during the selection process.

4.1.3. Selection

In GAs, the selection process prefers chromosomes with high fitness values to pass on to the next generation. This process allows the algorithm to converge quickly toward an acceptable solution. Roulette wheel selection is one of the selection operators used in this process, and it determines the probability of each chromosome being chosen based on its fitness value. As the name suggests, this method operates like a spinning roulette wheel, where the probability of each chromosome being selected is proportional to its fitness value. Mathematically, the selection probability $P(i)$ of a chromosome is expressed as its fitness value, fitness(i), divided by the sum of the fitness values of all chromosomes:

$$P(i) = \frac{\text{fitness}(i)}{\sum_{j=1}^{N} \text{fitness}(j)} \tag{16}$$

where i is the index of a chromosome in the population and N is the population size.

The main benefit of this approach is that it gives every chromosome some likelihood of selection, which would allow it to be passed on to the next generation without losing diversity, and that chromosomes with higher fitness are selected with a higher probability, which helps the algorithm find acceptable solutions quickly.

Because roulette wheel selection is stochastic, there is a risk that high-performing chromosomes may not be selected in a given generation. To mitigate this, the algorithm employs an elitist preservation strategy: the top e chromosomes from the previous generation (ranked by fitness) are unconditionally carried forward by replacing the lowest-fitness offspring. In this study, the elitism count is set to $e = \lceil \alpha_e \times \text{popSize} \rceil$, where α_e is the elitism rate (see Table 5), ensuring that the best-known solutions are never lost across generations.

4.1.4. Crossover

In crossover, two parent chromosomes are selected and their genes are recombined to create offspring chromosomes. This process intentionally alters values or sequences within the chromosome to increase diversity and reduce selection pressure, which is the tendency for the algorithm to favor certain genes over others, potentially leading to premature convergence. As shown in Fig. 3, the original chromosomes are composed of waypoint information for each robot, which includes the specific waypoints that each robot must visit and the sequence in which they are visited. The chromosomes are processed by combining the waypoint information, meaning the set of waypoints and their visitation sequence, for each robot into a single unified list or sequence entry for each chromosome.

As illustrated in Fig. 3, during chromosome processing, the offspring first inherits the first half of the waypoint sequence from the higher-scoring parent and then inherits the second half from the lower-scoring parent. If this results in duplicate genes in the offspring chromosome, the duplicate from the lower-scoring half is replaced with an unused gene, which is a gene not yet present in the offspring sequence. For instance, consider a high-scoring processed chromosome $(9, 13, 10, 14, 11, 15)$ and a low-scoring chromosome $(9, 14, 13, 11, 10, 15)$. The offspring would initially inherit $(9, 13, 10)$ from the high-scoring parent and $(11, 10, 15)$ from the low-scoring one, forming $(9, 13, 10, 11, 10, 15)$. However, the 10 gene is duplicated in this sequence. Thus, the duplicate 10 gene from the lower-scoring half is replaced by an unused gene, such as the 14 gene, resulting in $(9, 13, 10, 11, 14, 15)$. The crossover point is fixed at $\lfloor W/2 \rfloor$, where W is the total number of waypoints, so that each offspring inherits roughly equal portions from both parents. Next, the offspring directly inherit information regarding the number of waypoints each robot must traverse from their parents. Each gene in the chromosome represents a waypoint, and the sequence in the chromosome dictates the order in which the robots visit these waypoints. For example, if the offspring chromosome is $(9, 13, 10, 11, 14, 15)$ and the parental information for the number of waypoints is $(2, 2, 2)$ and $(3, 1, 2)$, respectively, these are restructured into $(9, 13)$, $(10, 11)$, $(14, 15)$, $(2, 2, 2)$ and $(9, 13, 10)$, (11), $(14, 15)$, $(3, 1, 2)$, indicating which waypoints each robot will traverse. Note that only two offspring are produced per crossover: one inherits the waypoint-count vector from the higher-scoring parent and the other from the lower-scoring parent. Although additional offspring could be generated by swapping the waypoint-count vectors, this role is deliberately delegated to the *Mutation of Waypoint Counts* operator (Section 4.1.5), which explores the waypoint-count space more broadly through random reassignment rather than being limited to recombinations of two parents.

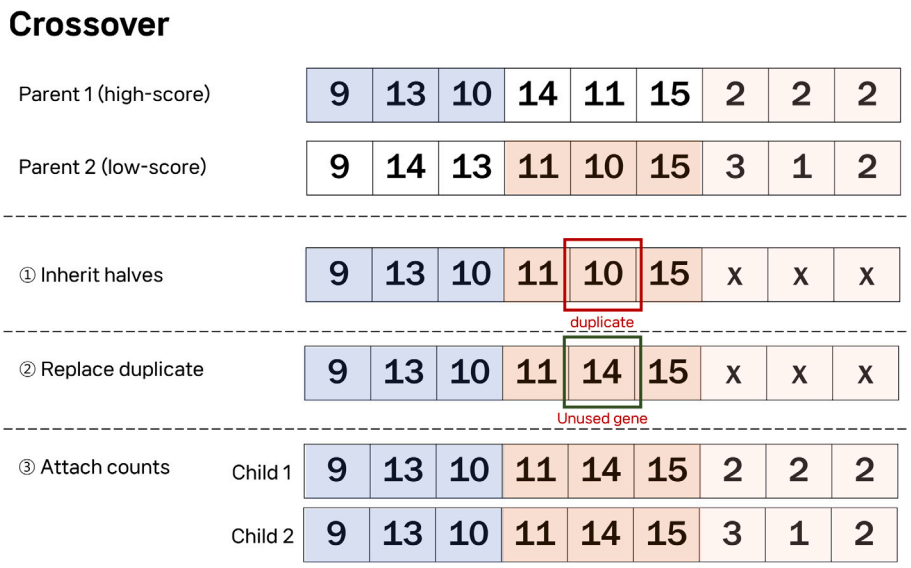


Fig. 3. Crossover operator in SGA-TS, illustrated with the parent chromosomes $(9, 13, 10, 14, 11, 15)/(2, 2, 2)$ and $(9, 14, 13, 11, 10, 15)/(3, 1, 2)$. Cell colors indicate parental origin: blue genes are inherited from the higher-scoring parent and beige genes from the lower-scoring parent, while pink cells hold the waypoint-count genes, which are excluded from sequence recombination (marked as x) and attached in the final step. Step 1: the offspring inherits the first half of the waypoint sequence from the higher-scoring parent and the second half from the lower-scoring parent, which may introduce a duplicate gene (red box). Step 2: the duplicate from the lower-scoring half is replaced with an unused gene (green box). Step 3: the waypoint-count vectors are attached, one from each parent, yielding two offspring with the same waypoint sequence but different count vectors. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

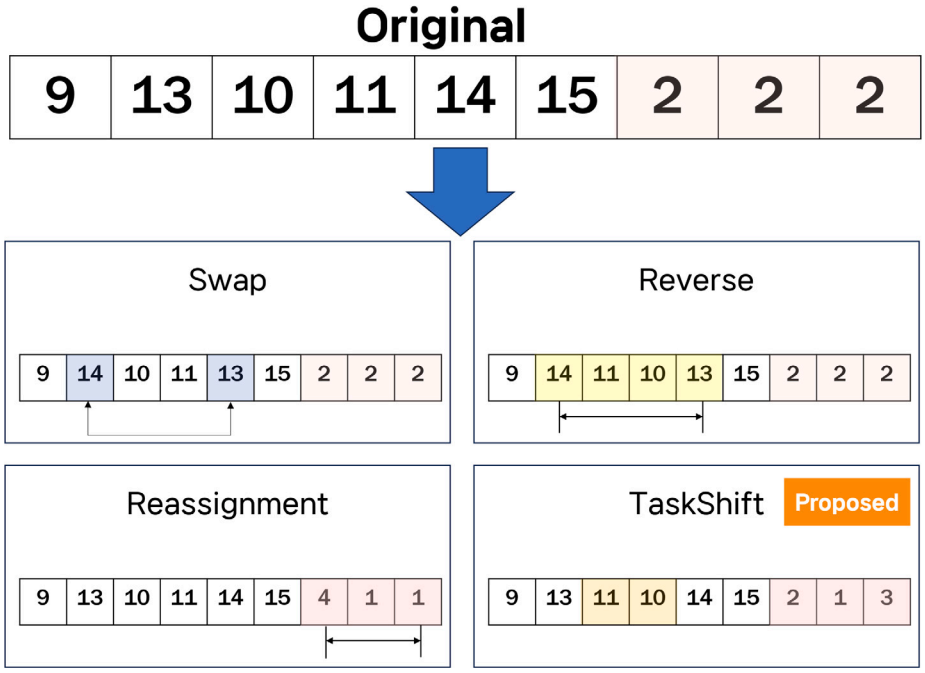


Fig. 4. The four mutation operators in SGA-TS, each applied to the original chromosome $(9, 13, 10, 11, 14, 15)/(2, 2, 2)$; shaded cells mark the genes modified by each operator. Swap exchanges the positions of two randomly chosen genes; Reverse inverts the order of a randomly chosen segment; Reassignment redistributes the waypoint-count genes (pink) among the robots; and TaskShift, highlighted as the proposed operator, reallocates waypoints between robots based on their current routes (Section 4.2).

4.1.5. Mutation

As illustrated in Fig. 4, before the algorithm produces the next generation, the offspring chromosomes are subjected to a mutation process, of which there are four key types:

- **Waypoint Swapping Mutation:** This involves randomly swapping the positions of two genes within the chromosome. For instance, in the chromosome $(9, 13, 10, 11, 14, 15)$, the genes 13 and 14 might switch places, which would give the chromosome $(9, 14, 10, 11, 13, 15)$ (see Fig. 4).
- **Waypoint Reversal Mutation:** A segment of the chromosome is reversed. This is done by first choosing a number between 2 and the chromosome's length to determine the number of genes to reverse. Then, a random starting point for the reversal is selected. For example, if the chromosome is $(9, 13, 10, 11, 14, 15)$ and four genes starting from position 2 (1-based indexing) are to be reversed, the new sequence would be $(9, 14, 11, 10, 13, 15)$ (see Fig. 4).
- **Mutation of Waypoint Counts:** This mutation alters the chromosome data that indicates the number of waypoints each robot should visit. The numbers of waypoints are randomly reassigned within the total waypoint count. For instance, the chromosome $(2, 2, 2)$ could be altered to $(1, 0, 5)$ or $(2, 3, 1)$.
- **TaskShift (Local Improvement):** Unlike the preceding stochastic mutations, TaskShift is a *deterministic* local improvement operator that reallocates waypoints among robots based on their current routes. Given the same chromosome, it always produces the same result. It is applied stochastically with probability p_{ts} to balance exploitation (local refinement) with exploration (maintaining population diversity). The specifics of the TaskShift algorithm are explained in Section 4.2.

The combination of a GA with a deterministic local improvement operator such as TaskShift is commonly referred to as a *memetic algorithm* (MA) [63,64]. In the MA framework, the GA provides global exploration of the search space, while the local search operator refines promising solutions. A key design choice in MAs is controlling how frequently the local search is invoked: applying it to every chromosome would yield faster convergence but at a prohibitive computational cost, whereas applying it too rarely would fail to exploit the local structure of the search landscape. In SGA-TS, TaskShift is triggered with a small probability ($p_{ts} = 0.05$), following standard MA practice [64], which strikes a balance between solution quality and computational efficiency.

The probability of each type of mutation was set to 5% in this study. After mutation, the elitist preservation described in Section 4.1.3 is applied: the top e chromosomes from the previous generation replace the worst e offspring (only when the elite fitness exceeds the offspring fitness), thereby preventing the loss of high-quality solutions.

4.2. Taskshift algorithm of SGA-TS

When the GA is used without TaskShift, unnecessary route costs can arise, as shown in Fig. 5. In the case of Fig. 5(a), R_1's route includes R2-2, R2-4, and R3-3, but because these are not assigned tasks for R_1, additional unnecessary costs are incurred for R_2 and R_3 to execute these tasks. TaskShift is designed to mitigate this path-level inefficiency through inter-route waypoint reassignment.

The TaskShift algorithm dynamically adjusts task assignments: when a robot encounters another robot's waypoint along its computed path, it may absorb that waypoint into its own task list, thereby reducing the donor's workload and improving route efficiency. For example, the transition from Figs. 5(a) to 5(b) shows how R_1 absorbs waypoints R2-2, R2-4, and R3-3, improving overall route efficiency.

TaskShift is related to established local improvement operators in vehicle-routing heuristics, including 2-opt, relocate, Or-opt, and cross-exchange [60,61]. We do not claim TaskShift to be a new family of local search. Rather, its role in SGA-TS is a specific path-aware inter-route reassignment operator: candidate waypoints are generated from the receiver robot's JPS-computed cell-level traversal path and are then filtered by a bottleneck-first rule. Table 4 summarizes this distinction.

Table 4
Relation between TaskShift and established local improvement operators.

Aspect	Established VRP improvement operators	TaskShift in SGA-TS
Representative moves	2-opt, relocate, Or-opt, and cross-exchange; route-sequence refinements.	Path-aware reassignment embedded in the GA mutation stage.
Candidate source	Route nodes, edges, vertices, or segments selected from the chromosome sequence.	Waypoints encountered on the receiver robot's JPS cell-level traversal path.
Path information	Uses scalar edge costs; intermediate traversed cells are not candidate triggers.	Uses traversed JPS cells to identify waypoints already lying along another robot's path.
Filtering and role	Usually accepted by local route-cost improvement; general route refinement.	Bottleneck-first transfer from the current bottleneck to a lower-cost receiver for flowtime co-improvement under a makespan-first GA.

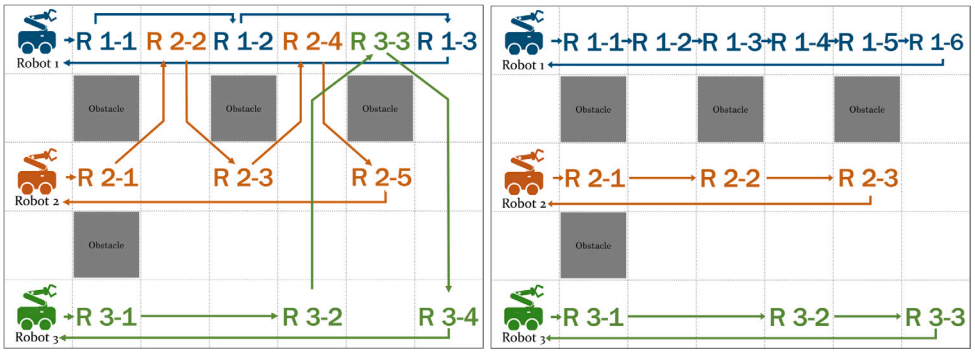
TaskShift Algorithm (Algorithm 2) proposed in this study is designed to efficiently update the travel paths of the robots. This algorithm takes as input *total robot*, which is a list of all robots, *total waypoint*, which is a collection of the waypoints that all robots must traverse, and *total start point*, which is a collection of the start points of all robots. In the first step, an empty list *path* is initialized for each robot, along with a parallel list *seg* that records the segment index of each cell. The segment index indicates which leg of the journey (e.g., depot→waypoint$_1$, waypoint$_1$→waypoint$_2$, ...) each path cell belongs to.

The algorithm then iterates through each waypoint in the robot's list to construct the cell-by-cell path using JPS. The starting position for the first waypoint is set to the robot's start location, and for subsequent waypoints, the starting position is set to the previous waypoint in the list. This iterative approach is used because each waypoint's role changes dynamically from being a destination to becoming a starting point for the next leg of the journey, which ensures continual path progression for the robot. After visiting all waypoints, a return segment from the last waypoint back to the depot is appended to complete the round-trip path.

After it completes the path calculation, the algorithm computes the completion time of every robot and identifies the current bottleneck. A snapshot of $(cell,\ segment\text{-}index)$ pairs is then taken from the computed path (excluding the last cell), and the algorithm iterates over this fixed snapshot. For each cell, if the cell is a waypoint that needs to be visited (a member of *total waypoint*) and it is not already in the robot's waypoint list, the cell is considered for transfer. The transfer is permitted only when the donor robot is the current bottleneck (i.e., has the maximum completion time) and the receiving robot's completion time is strictly less than the donor's. This *bottleneck-first* rule selectively moves tasks away from the most overloaded robot to a less loaded one while retaining the makespan-first objective. The transferred waypoint is inserted at the position indicated by its snapshotted segment index, placing it at the natural point along the robot's route where the cell was encountered. After each transfer, all robot times and the bottleneck designation are recalculated, so that subsequent transfers target the updated bottleneck. Because the iteration proceeds over the fixed snapshot, the segment indices used for insertion remain consistent regardless of how the waypoint list changes during the scan.

The transfer rule can be written more formally as follows. Let $\mathcal{W}_k(x)$ denote the waypoint set assigned to robot k in solution x, and let the current bottleneck robot be

$$b(x) = \arg\max_{k \in D} \mathrm{Cost}_k(x). \tag{17}$$



(a) Path before applying the TaskShift operator. (b) Path after applying the TaskShift operator.

Fig. 5. Example of path-aware task reassignment by TaskShift. When a robot's computed path passes through waypoints assigned to other robots, TaskShift can reassign those waypoints to reduce redundant travel.

For a candidate receiver robot r, TaskShift constructs the cell-level traversal path

$$P_r(x) = \bigcup_{p=0}^{m_r-1} \mathrm{JPSPath}(v_{r,p}, v_{r,p+1}), \tag{18}$$

where JPSPath$(\cdot)$ returns the sequence of grid cells traversed between two consecutive route nodes. A waypoint v is transferred from donor robot d to receiver robot r only if

$$v \in P_r(x) \cap \mathcal{W}_d(x), \quad d = b(x), \quad \mathrm{Cost}_r(x) < \mathrm{Cost}_d(x). \tag{19}$$

The resulting reassignment is denoted by $x' = \mathcal{T}_{d\to r}(v; x)$. Algorithm 2 does not impose a formal acceptance test such as $M(x') \le M(x)$ after every transfer; instead, it uses the bottleneck-first rule as a heuristic filter and recomputes robot costs after each accepted transfer. In our preliminary validation, this rule produced no observed makespan degradation across 30 independent runs (Table 7).

The TaskShift algorithm offers several key advantages. First, it can reduce the makespan for the MRTA by optimizing the travel paths of the robots. For example, in Fig. 5(a), the robot with the longest travel distance, R_3, moves 18 units, which represents the makespan of the task allocation. However, in Fig. 5(b), R_1 and R_3 have the longest travel distances of just 12 units each, indicating a reduced makespan in this example. Second, the overall flowtime is reduced. In Fig. 5(a), the total travel distances for the robots were 12, 14, and 18 units, which sums to 44 units. In contrast, Fig. 5(b) shows distances of 12, 10, and 12 units, for a total travel distance of 34 units. The core principle of the TaskShift algorithm (Algorithm 2) is for a robot to incorporate tasks from other robots into its workload when those tasks are positioned along the robot's route. This integration is designed to add little or no extra route cost to the receiver because the transferred waypoint has already appeared on the receiver's computed path. Meanwhile, the robots whose tasks are taken away may benefit from a more efficient route.

The bottleneck-first transfer rule in Algorithm 2 governs the priority and conflict avoidance of TaskShift. Because robots are processed sequentially in index order and a task transfer is permitted only when the donor is the current bottleneck, no two robots can simultaneously claim the same waypoint. Furthermore, the dynamic recalculation after each transfer prevents excessive task removal from any single robot: once a transfer changes the bottleneck identity, subsequent transfers are directed at the new bottleneck.

The SGA-TS algorithm (Algorithm 3) operates as follows. The algorithm first initializes the operational parameters — number of robots (n_{robots}), tasks (n_{tasks}), maximum generations (maxGen), population size (popSize), and mutation probabilities (p_{swap}, p_{rev}, p_{cnt}, p_{ts}) — and generates an initial population comprising popSize − 1 random chromosomes and one greedy seed chromosome (Line 1). The fitness of each chromosome is then evaluated (Line 2).

Table 5
Genetic algorithm parameter settings.

Parameter	Description and purpose	Setting
Population size	Number of chromosomes maintained in each generation.	100
Number of generations	Total iterations over which the GA evolves.	500
Elitism rate (α_e)	Fraction of top solutions preserved for the next generation; $e = \lceil \alpha_e \times \text{popSize} \rceil$.	0.05
Mutation probability	Probability of introducing random changes in each chromosome to avoid premature convergence.	5%
Fitness function	Objective function defined as minimizing makespan (see Eq. (13)).	min $M(x)$

Within each generation, parent chromosomes are selected via roulette wheel selection and recombined through crossover. Each offspring undergoes stochastic mutation: waypoint swapping, waypoint reversal, waypoint-count reassignment, and TaskShift local improvement. These operators introduce diversity to prevent premature convergence while TaskShift refines solutions through deterministic local search. After evaluating offspring fitness, elitist preservation replaces the worst e offspring with the top e chromosomes from the previous generation when the elite fitness exceeds the offspring fitness. The stored best chromosome is then updated only when the best offspring has a lower makespan than the current stored best, i.e., $M(x^{\text{off}}_{\text{best}}) < M(x^{\text{stored}}_{\text{best}})$.

Upon reaching maxGen generations, the algorithm returns the stored best chromosome, representing a near-optimal task allocation. Compared with the basic GA (Algorithm 1), which relies on a single mutation operator, SGA-TS introduces a richer set of genetic operators alongside elitist preservation, broadening the search space exploration while preventing the loss of high-quality solutions across generations.

4.3. Computational complexity analysis

This section analyzes the computational complexity of SGA-TS, with particular attention to the cost of the TaskShift operator. Let n denote

Algorithm 2 TaskShift Algorithm

Require: *total_robot* (List of all robots), *total_waypoint* (Collection of waypoints all robots must traverse), *total_starting* (Collection of start point of all robots)
Ensure: Updated Robots' Waypoints
for each *robot* in *total_robot* **do**
  if $|robot.waypoints| = 0$ **then**
    continue ▷ Skip robots with no assigned waypoints
  end if
  $i \leftarrow 0$
  $path \leftarrow$ empty list
  $seg \leftarrow$ empty list ▷ Segment index for each cell in *path*
  while $i <$ length of $robot.waypoints$ **do**
    if $i == 0$ **then**
      $start \leftarrow robot.start$ in *total_starting* ▷ Start point of the robot
    else
      $start \leftarrow robot.waypoints[i-1]$
    end if
    $path.extend(\text{JPS}(grid, start, robot.waypoints[i]))$ ▷ Calculate path using JPS
    $seg.extend(i$ for each appended cell$)$
    $i \leftarrow i + 1$
  end while
  $path.extend(\text{JPS}(grid, robot.waypoints[\text{last}], robot.start))$ ▷ Return to depot
  $seg.extend(i$ for each appended cell$)$
  $T \leftarrow$ ComputeAllRobotTimes($total_robot$) ▷ Completion time of each robot
  $T_{max} \leftarrow \max(T)$ ▷ Current bottleneck time
  $pathCells \leftarrow [(path[j],\ seg[j])$ for $j = 0, \ldots, |path| - 2]$ ▷ Snapshot (cell, segment index) pairs
  for each $(cell,\ s)$ in $pathCells$ **do**
    if $cell \in total_waypoint$ and $cell \notin robot.waypoints$ **then**
      for each *donor* in *total_robot* **do**
        if $donor \neq robot$ and $cell \in donor.waypoints$ **then**
          if $T[donor] = T_{max}$ **and** $T[robot] < T[donor]$ **then** ▷ Bottleneck-first rule
            $donor.waypoints.remove(cell)$
            $insert_pos \leftarrow \min(s,\ |robot.waypoints|)$
            $robot.waypoints.insert(insert_pos, cell)$
            $T \leftarrow$ ComputeAllRobotTimes($total_robot$) ▷ Recalculate
            $T_{max} \leftarrow \max(T)$
          end if
        end if
      end for
    end if
  end for
end for
return Updated Robots' Waypoints

Algorithm 3 SGA-TS Algorithm for Multi-Robot Task Allocation

Initialize parameters: number of robots (n_{robots}), number of tasks (n_{tasks}), maximum generations (maxGen), population size (popSize), mutation probabilities (p_{swap}, p_{rev}, p_{cnt}, p_{ts}), elitism count $e = \lceil \alpha_e \times \text{popSize} \rceil$.
Fitness: fitness$(x) = 1/M(x)$; therefore, lower makespan implies higher fitness.
Ensures: Near-optimal task allocation among robots.
Generate initial population: popSize − 1 chromosomes with random genes and 1 chromosome using a greedy algorithm.
Evaluate fitness of each chromosome in the initial population.
gen ← 1 ▷ Current generation number
Store the chromosome with the highest fitness as $x_{\text{best}}^{\text{stored}}$.
while gen ≤ maxGen **do**
  Select parent chromosomes using roulette wheel selection.
  Perform crossover to generate offspring chromosomes.
  for each offspring chromosome **do**
    $r_{swap}, r_{rev}, r_{cnt}, r_{ts} \leftarrow$ Random number between 0 and 1
    if $r_{swap} \leq p_{swap}$ **then**
      Apply waypoint swapping mutation.
    end if
    if $r_{rev} \leq p_{rev}$ **then**
      Apply waypoint reversal mutation.
    end if
    if $r_{cnt} \leq p_{cnt}$ **then**
      Apply mutation of waypoint counts.
    end if
    if $r_{ts} \leq p_{ts}$ **then**
      Apply TaskShift local improvement.
    end if
  end for
  Evaluate fitness of new offspring chromosomes.
  Let $x_{\text{best}}^{\text{off}}$ be the best chromosome among the new offspring.
  Select top e chromosomes from the **previous** generation (elites).
  Replace the worst e offspring with elites where elite fitness > offspring fitness. ▷ Elitism
  if $M(x_{\text{best}}^{\text{off}}) < M(x_{\text{best}}^{\text{stored}})$ **then**
    $x_{\text{best}}^{\text{stored}} \leftarrow x_{\text{best}}^{\text{off}}$
  end if
  gen ← gen + 1
end while
return $x_{\text{best}}^{\text{stored}}$

the number of robots, W the number of waypoints, $P = n + W$ the total number of points (depots and waypoints), G the number of grid cells ($R \times C$ for an R-row, C-column grid), S the population size, and $G_{\max}$ the maximum number of generations.

4.3.1. Fitness evaluation

Computing the fitness of a single chromosome requires determining each robot's travel cost. This involves obtaining the JPS travel cost for each consecutive pair of points along the robot's route. Across all n robots, there are at most $W + n$ such pairs (each robot with m_k waypoints requires $m_k + 1$ segment calls, totaling $\sum_k (m_k + 1) = W + n$). If scalar pairwise costs are not already cached, each JPS call operates on the grid in $O(G)$ time in the worst case. Thus, the no-cache worst-case cost of evaluating a single chromosome is $O((W + n) \cdot G)$, and evaluating the entire population costs $O(S \cdot (W + n) \cdot G)$ per generation. A scalar distance matrix may be cached for repeated cost lookup, but TaskShift additionally requires the cell-by-cell traversal path for the route segments it inspects, not only the scalar cost.

4.3.2. TaskShift Local Improvement (Algorithm 2)

The TaskShift operator constructs each robot's cell-by-cell path (including the return-to-depot segment) by calling JPS for each segment, costing $O((W + n) \cdot G)$ per chromosome. A snapshot of the path cells and their segment indices is then taken, and the operator iterates over this snapshot. Upon each task transfer, all robot completion times are recomputed, which requires additional JPS calls to recalculate the affected routes. In the worst case, the per-chromosome cost of TaskShift is:

$$T_{\text{TS}} = O((W + n) \cdot G \cdot (1 + t)), \tag{20}$$

where t is the number of task transfers performed. Since each transfer moves one waypoint, $t \leq W$.

4.3.3. Per-generation cost

Within each generation, the following operations are performed: selection via roulette wheel in $O(S)$; crossover in $O(S \cdot W)$; standard mutations (swap, reverse, waypoint count) in $O(S \cdot W)$; TaskShift local improvement with probability p_{ts}; and fitness evaluation for all chromosomes. The dominant cost is the JPS calls required for fitness

evaluation and TaskShift. Because TaskShift is applied with probability p_{ts} (a small constant; $p_{ts} = 0.05$ in our experiments) and the number of transfers t is bounded by W, the expected per-chromosome TaskShift cost is $p_{ts} \cdot O((W+n) \cdot G \cdot (1+t))$. In the amortized analysis, the $(1+t)$ factor is absorbed into the leading $(W+n)$ term since $t \leq W$, yielding the per-generation cost:

$$T_{\text{gen}} = O\big(S \cdot (W+n)^2 \cdot G\big). \tag{21}$$

4.3.4. Overall complexity

The total time complexity of SGA-TS is:

$$T_{\text{total}} = O\big(G_{\max} \cdot S \cdot (W+n)^2 \cdot G\big). \tag{22}$$

The dominant cost factor is the repeated JPS calls for distance computation across generations, fitness evaluations, and TaskShift local improvement operations.

5. Simulation results and discussion

Our empirical evaluation consists of three experimental parts (Sections 5.1–5.3), a runtime analysis (Section 5.4), and an overall discussion (Section 5.5).

SGA-TS is compared with six baseline algorithms (detailed in Section 2): SMP-GA [6] (multi-population seeding), SHC-GA [52] (hybrid chromosome seeding), GA-2OPT [40] (GA with 2-Opt local search), IMC-GA [51] (interpretable multi-chromosome GA), ACO [46] (dual-pheromone ant colony), and MO-MOEA [24] (maximin Pareto multi-objective EA). MO-MOEA is included to compare SGA-TS against a method that explicitly handles both makespan and flowtime objectives.

We also examined a representative learning-based MRTA method, CapAM [50], because learning-based policies have recently been explored for MRTA. CapAM was selected as a publicly available graph reinforcement learning baseline; however, after adapting it to our obstacle-aware grid-based min–max routing setting, the model did not converge and produced deterministic noncompetitive allocations. We therefore exclude CapAM from the primary ranking tables: comparing against a non-converged learning model would be unfair and would overstate the apparent performance gap. The primary comparisons below are therefore restricted to methods that completed their optimization process under the tested settings.

The experimental environment for Sections 5.1 and 5.2 is shown in Fig. 6, with six layouts and three robot-count settings. Layouts (b) and (c) adopt the layout from [6], whereas Layout (d) uses the layout from [45,65]. In contrast, Layouts (a), (e), and (f) are grid-based maps devised by the authors to demonstrate how the algorithm operates under various conditions. Layouts (a)–(c) are characterized by either a complete absence of obstacles or the presence of only minor obstacles that do not significantly hinder robot movement. In contrast, Layouts (e)–(f) feature a central space composed entirely of obstacles, necessitating inefficient detours, or widely distributed obstacles that make diagonal movement impossible. The number of waypoints (W) per layout is: $W = 90$ for Layouts (a), (b), (d), (e), and (f), and $W = 98$ for Layout (c). The number of robots used in all experiments is summarized in Table 6. When five robots are used, only the lime-colored robots are used; for ten robots, both lime and green-colored robots are used; and for twenty robots, lime, green, and red-colored robots are used.

The experimental setting for this study was configured as follows. The hardware platform comprised a CPU (RYZEN 3900X) and RAM (DDR4 32 GB). The robot's mobility was programmed to include eight possible directions—four cardinal (up, down, left, right) and four diagonal directions. The operating system was Windows 10, and the experiments were conducted using Python version 3.5.

In addition to the hardware and software specifications, a standard set of Genetic Algorithm (GA) parameters was employed for all experiments to ensure consistency and reproducibility. The GA framework was used to optimize the task allocation, with a particular focus on maintaining diversity in the solution population and preventing premature convergence. Table 5 summarizes the key GA parameters, including the population size, number of generations, elitism rate, mutation probability, and the fitness function defined as minimizing makespan (Eq. (13)). As a preliminary validation of the TaskShift operator, Table 7 shows that out of 30 independent runs, 17 achieved an improved makespan while the remaining 13 retained the same makespan, with no observed degradation in this validation set.

Table 6
Number of robots used in different environments: In Sections 5.1 and 5.2, experiment with different numbers of robots, with different colors for each number of robots.

	Lime	Green	Red
5 robots	O		
10 robots	O	O	
20 robots	O	O	O

Table 7
Preliminary validation of the TaskShift operator: number of experiments (out of 30) in which makespan improved, remained unchanged, or worsened.

	Improved makespan	Same makespan	Worsened makespan
Number of experiments	17	13	0

5.1. Verifying the efficacy in optimizing makespan

Each experiment was conducted 30 times; results are summarized in Table 8 and Fig. 7.

5.1.1. Comparison of SGA-TS with baseline algorithms

Our experimental study compared SGA-TS with six baseline MRTA methods — SMP-GA, IMC-GA, SHC-GA, GA-2OPT, ACO, and MO-MOEA — across 18 diverse scenarios. One-way ANOVA confirmed statistically significant differences among the seven algorithms in every scenario (all $p < 0.01$). Across all 18 scenarios, SGA-TS achieved the lowest mean makespan.

Both SMP-GA and SHC-GA benefit from heuristic seeding; SHC-GA further employs a hybrid chromosome structure to integrate high-quality initial solutions. However, despite these advantages, SGA-TS consistently outperformed SHC-GA as well as SMP-GA by dynamically adjusting task assignments through the TaskShift algorithm during the evolutionary process. Similarly, GA-2OPT, which refines initial solutions using a 2-Opt local search, and IMC-GA, which utilizes an interpretable multi-chromosome framework, resulted in higher average makespans compared to SGA-TS.

Overall, SGA-TS achieved the lowest average makespan in all 18 scenarios, with average improvements ranging from 17.27% (over SMP-GA) to 48.12% (over IMC-GA). The improvements over ACO and MO-MOEA averaged 23.27% and 26.36%, respectively.

5.1.2. Layout complexity analysis

To examine how obstacle complexity affects the relative performance of SGA-TS, Fig. 8 presents the per-layout makespan improvement percentages of SGA-TS over each baseline. Each per-layout value was computed as the mean of scenario-wise improvements across the three robot counts (5, 10, and 20).

In layout (a), which has few obstacles, SGA-TS achieved modest improvements of 7.0% over SMP-GA, 10.6% over SHC-GA, 19.0% over ACO, and 16.7% over MO-MOEA, while the improvements over IMC-GA (42.0%) and GA-2OPT (39.2%) were larger. In layouts (e) and (f), which contain both extensive obstacles and dead-end structures, SGA-TS achieved the largest gains: 18.8% and 46.1% over SMP-GA, 38.8%

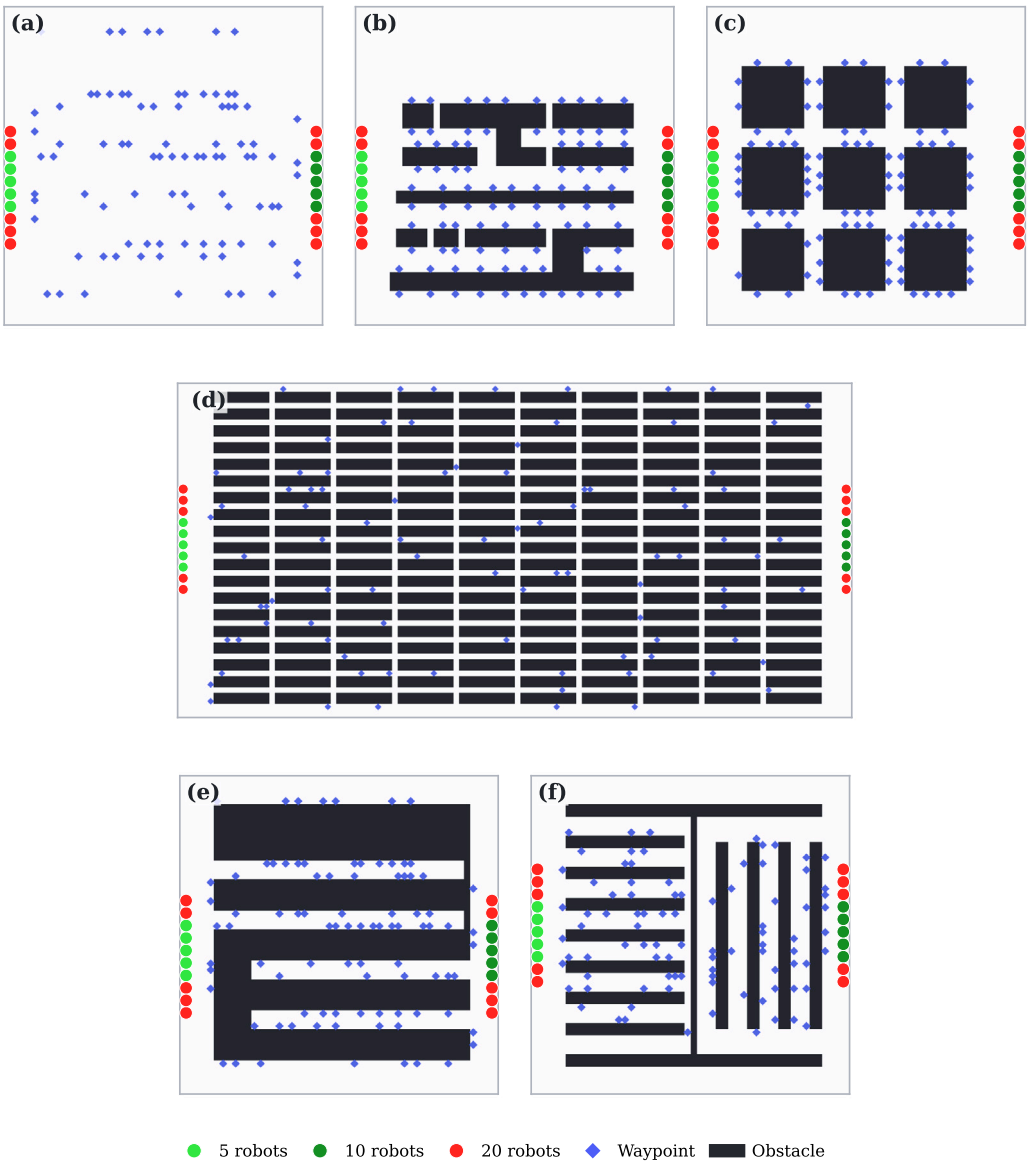


Fig. 6. Experimental layouts used in the benchmark scenarios. Blue diamonds denote waypoints, colored circles denote robot depots for the 5-, 10-, and 20-robot settings, and black regions denote obstacles. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

and 40.3% over SHC-GA, and 54.3% and 53.7% over IMC-GA, respectively. The intermediate layouts showed improvements between these extremes, though the progression was not strictly monotonic across layouts (b)–(d).

The improvement over ACO does not follow this trend. The largest improvement over ACO was observed in layout (b) at 31.6%, whereas the gap narrowed to 18.0% in layout (f). This may indicate that ACO's pheromone-based search is relatively more competitive in highly structured environments with narrow corridors, although SGA-TS still achieved the lowest makespan in all cases.

Overall, the average improvement across all six baselines was 22.4% in layout (a) and rose to 41.0% in layout (f), suggesting that the seeding strategy and TaskShift mechanism of SGA-TS yield larger gains in environments with higher obstacle complexity, particularly in layouts (e) and (f).

5.1.3. Robot-to-waypoint ratio analysis

To assess the effect of the robot-to-waypoint ratio on the performance of SGA-TS relative to the baseline algorithms, we aggregated the average makespan values across all layouts (a)–(f) for experiments conducted with 5, 10, and 20 robots. For each robot count, we first summed the average makespan values from the six layouts and then computed the overall average.

According to Table 8 and Fig. 9, for experiments with 5 robots, SGA-TS achieved the following makespan improvements compared to other algorithms: approximately 11.49% over SMP-GA, 48.77% over IMC-GA, 15.28% over SHC-GA, and 36.45% over GA-2OPT.

For experiments with 10 robots, SGA-TS reduced the makespan by around 21.82% compared to SMP-GA, 52.34% compared to IMC-GA, 32.37% compared to SHC-GA, and 53.25% compared to GA-2OPT.

For experiments with 20 robots, SGA-TS achieved improvements of about 18.49% over SMP-GA, 43.26% over IMC-GA, 24.73% over SHC-GA, and 51.68% over GA-2OPT.

These results suggest that the relative performance improvement offered by SGA-TS depends on the robot-to-waypoint ratio. With only 5 robots, the limited number of agents means that each robot is responsible for a larger set of nearby waypoints. This reduces the opportunity for dynamic task reallocation to significantly improve the

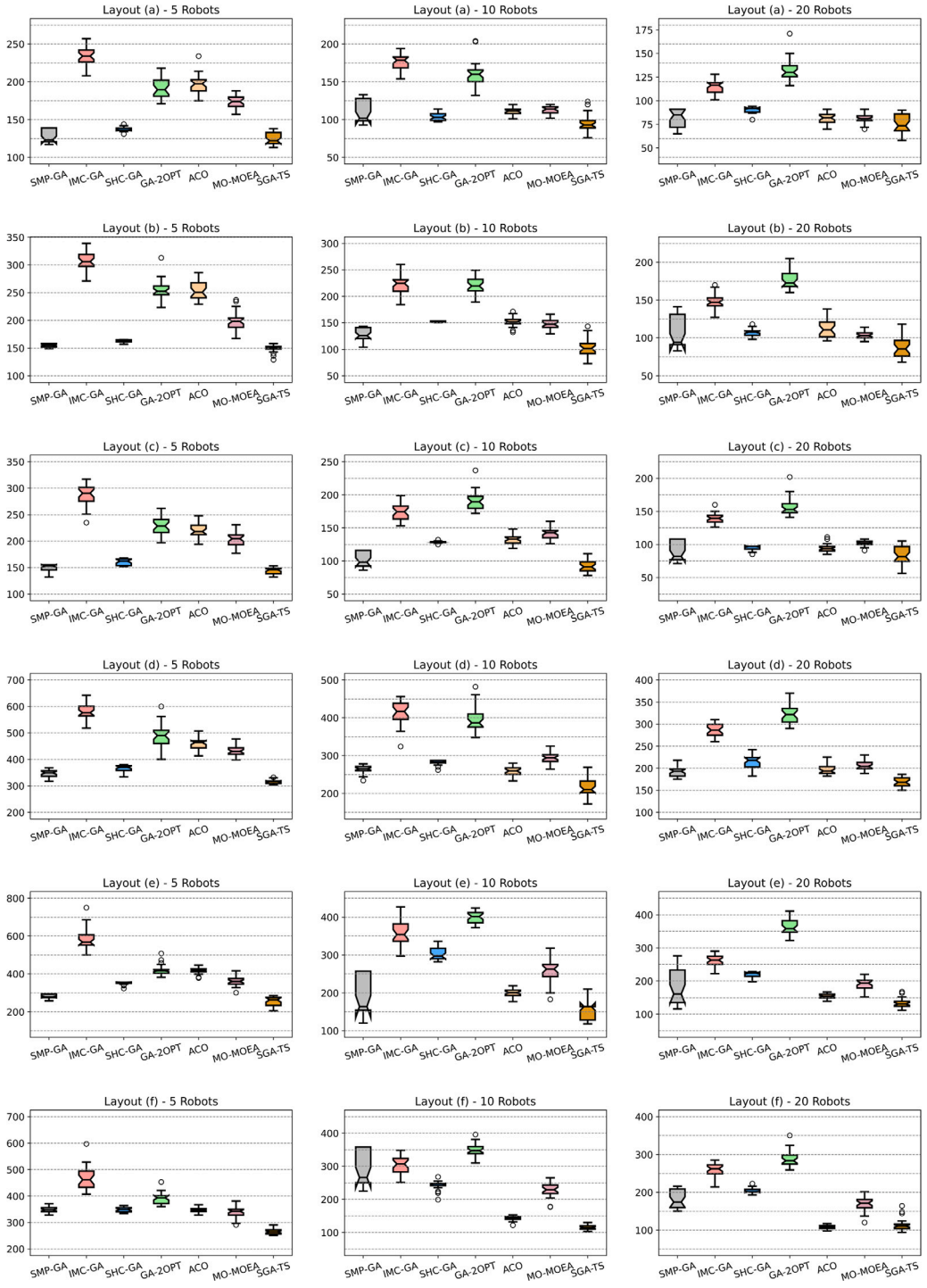


Fig. 7. Boxplot of makespan. In each boxplot, the box spans the interquartile range (Q1–Q3), the horizontal line denotes the median, whiskers extend to 1.5× IQR, and points beyond the whiskers indicate outliers.

overall makespan, resulting in only modest improvements. However, with 20 robots, the increased number of agents results in each robot being assigned fewer tasks, which reduces the potential benefits of dynamic task reallocation. As robots have fewer waypoints to visit, the overall gains from optimizing their routes become less significant, leading to a somewhat lower improvement compared to some baseline algorithms like SMP-GA. Overall, these findings suggest that an optimal balance between the number of robots and the number of tasks is critical for maximizing the effectiveness of SGA-TS. This diminished improvement at higher robot counts stems from the bottleneck-first transfer rule in TaskShift: with 20 robots each carrying only ~4.5 waypoints, the bottleneck robot's cost is already low and completion times across robots converge, leaving fewer valid transfer opportunities. A comparison with the flowtime trend, where the improvement drop is less pronounced, is provided in Section 5.5.

5.2. *Verifying the efficacy in improving flowtime*

This experiment evaluates the flowtime achieved by SGA-TS compared to the same six baselines, across the same six layouts and

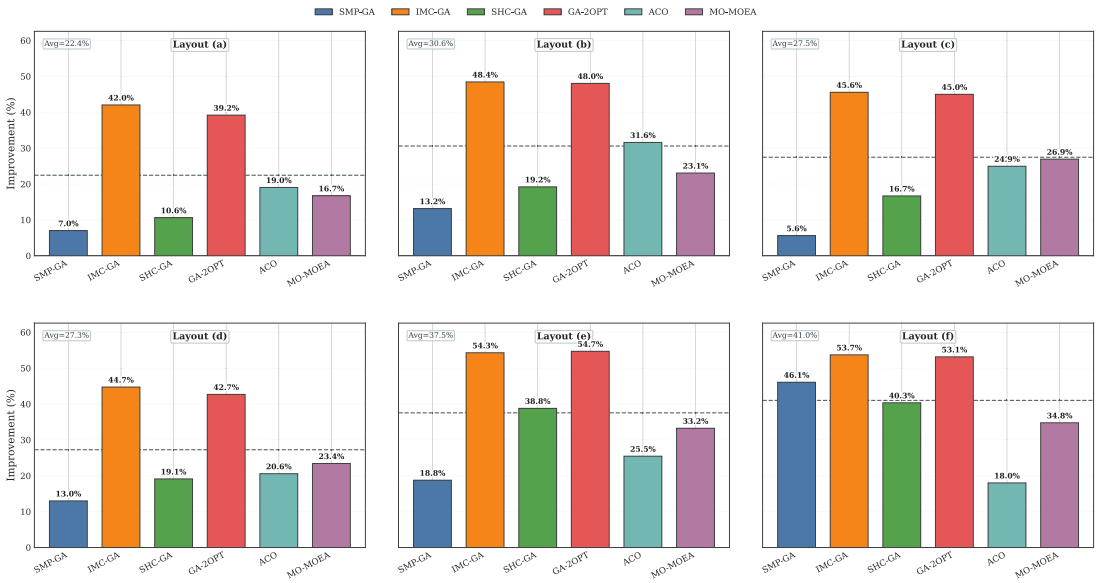


Fig. 8. Per-layout makespan improvement (%) of SGA-TS over each baseline algorithm. Each bar represents the mean improvement across three robot counts (5, 10, and 20).

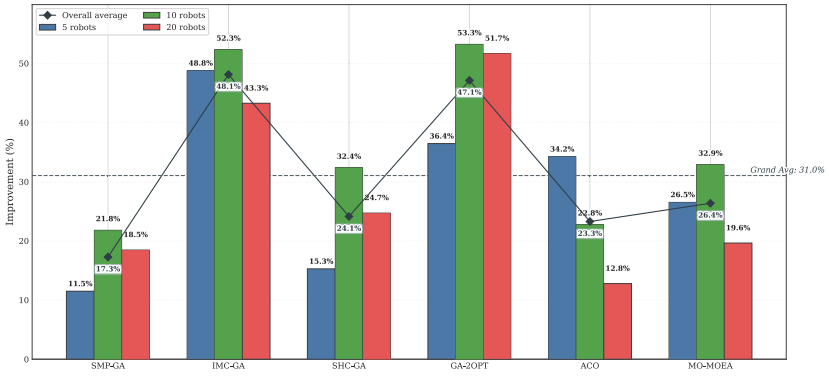


Fig. 9. Makespan improvement (%) of SGA-TS over baseline algorithms by number of robots.

Table 8
Experiment result of makespan.

Layout	#Robots	SMP-GA		IMC-GA		SHC-GA		GA-2OPT		ACO		MO-MOEA		SGA-TS	
		Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev
	5	126.93	8.59	234.21	11.86	137.03	2.93	191.10	13.32	196.43	12.02	173.43	8.72	**124.41**	8.33
(a)	10	108.69	14.27	175.62	10.76	103.62	5.02	159.83	16.28	110.97	4.69	113.30	4.96	**95.86**	9.35
	20	81.86	9.82	114.69	6.97	89.48	4.59	132.48	10.65	81.50	5.61	81.23	4.65	**75.93**	10.35
	5	154.34	3.05	305.41	18.47	162.38	2.01	252.97	18.57	253.97	16.35	198.03	18.13	**149.55**	6.37
(b)	10	127.31	11.84	223.31	18.63	151.83	0.87	220.21	15.69	152.37	9.02	147.03	9.18	**102.93**	16.14
	20	106.24	20.55	147.59	9.64	106.66	4.77	175.86	10.75	111.63	11.71	103.13	4.78	**87.97**	14.36
	5	150.03	5.89	286.79	19.45	160.55	6.00	229.31	17.71	219.23	13.37	203.53	13.90	**144.28**	5.45
(c)	10	101.79	10.24	174.97	12.27	128.66	1.93	191.24	13.80	131.83	6.93	140.97	7.58	**92.48**	9.56
	20	87.28	14.31	139.45	7.74	95.10	2.95	156.14	12.66	94.07	6.42	101.43	3.92	**83.90**	13.90
	5	347.53	11.37	580.55	26.95	364.76	13.86	487.59	43.84	460.13	22.23	430.73	18.08	**315.30**	6.97
(d)	10	264.23	9.22	412.48	30.21	282.07	6.01	396.21	30.27	258.87	12.07	293.47	16.06	**217.03**	25.00
	20	191.97	10.45	287.24	14.00	213.55	14.32	322.17	20.51	197.00	11.47	204.90	10.29	**169.20**	10.54
	5	281.07	10.67	585.45	56.47	351.03	6.90	417.69	24.93	417.83	17.69	361.23	24.32	**255.90**	24.76
(e)	10	192.31	51.73	358.62	35.15	302.90	16.77	398.90	15.37	200.80	10.62	256.53	28.62	**149.62**	24.19
	20	180.90	53.46	261.79	18.10	220.76	8.23	364.55	23.24	154.10	6.84	190.40	18.15	**135.41**	15.85
	5	457.37	28.91	467.31	42.29	349.17	9.05	390.00	19.96	346.93	9.66	338.87	19.63	**266.17**	10.22
(f)	10	233.40	18.19	301.41	28.04	243.00	12.56	346.97	19.35	142.07	6.27	229.00	21.15	**114.90**	7.22
	20	208.20	28.33	258.34	18.80	204.10	6.35	288.83	18.74	128.10	5.08	169.03	18.32	**113.20**	14.97

Table 9
Experiment result of flowtime.

Layout	#Robots	SMP-GA		IMC-GA		SHC-GA		GA-2OPT		ACO		MO-MOEA		SGA-TS	
		Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev
(a)	5	699.62	163.64	1116.31	62.38	**547.97**	16.54	737.28	40.52	947.47	60.67	851.07	44.75	592.52	110.62
	10	1041.86	189.59	1456.69	94.77	841.14	17.66	1031.17	52.23	1008.43	48.31	1086.40	55.45	**648.00**	106.37
	20	1532.38	309.24	1764.90	111.01	**1248.66**	57.79	1532.31	103.67	1338.23	80.69	1456.30	71.59	1265.45	120.50
(b)	5	858.97	155.35	1466.59	95.07	743.52	2.55	1018.55	55.91	1225.63	83.12	967.03	87.26	**740.69**	48.75
	10	1290.55	293.98	1844.28	159.56	1251.31	26.76	1414.72	97.20	1392.97	91.43	1408.73	91.18	**907.07**	137.32
	20	1849.66	410.42	2205.72	159.88	1766.93	133.13	2004.41	86.02	1777.10	182.17	1858.17	94.69	**1234.79**	288.29
(c)	5	816.93	146.27	1373.28	100.60	**714.66**	19.24	884.83	29.44	1054.37	63.01	996.60	68.18	725.59	83.42
	10	1097.59	348.53	1551.69	102.20	1140.86	17.83	1235.34	77.62	1212.27	76.16	1352.17	82.46	**872.24**	99.77
	20	1594.76	420.78	2169.48	147.04	**1215.59**	49.56	1826.21	98.09	1544.33	102.69	1835.00	94.09	1339.72	172.41
(d)	5	1869.90	330.10	2792.83	140.79	1641.24	37.61	1750.97	137.35	2230.23	111.99	2117.13	90.14	**1614.03**	59.44
	10	2799.72	617.69	3403.97	303.39	2169.93	47.51	2524.14	184.67	2382.90	121.18	2824.30	164.54	**2101.86**	219.25
	20	3240.00	859.60	4383.66	342.24	2969.93	124.09	3837.17	280.41	3261.63	150.78	3725.20	214.57	**2615.69**	81.43
(e)	5	1430.69	156.23	2710.52	268.36	1565.76	35.72	1694.10	157.31	2003.77	89.41	1754.93	123.24	**1204.90**	153.86
	10	1985.76	801.76	2859.76	347.05	1776.93	63.87	2512.59	225.09	1758.27	91.23	2401.80	289.30	**954.59**	147.72
	20	2898.55	887.00	3723.72	331.55	1499.52	72.62	3974.41	236.26	2304.57	154.28	3194.23	322.07	**1015.03**	99.47
(f)	5	1581.55	243.70	2197.97	211.74	1569.69	25.69	1532.00	117.44	1698.03	46.03	1664.43	98.31	**1326.00**	123.67
	10	1898.93	831.39	2468.10	295.09	1569.90	157.27	2250.10	223.24	1280.03	51.24	2166.37	205.88	**1119.07**	177.24
	20	3161.14	770.28	2971.24	277.49	2135.10	154.33	3428.14	241.34	1708.93	80.73	2702.67	404.88	**1379.41**	297.45

three robot counts (Table 6). Each setting was repeated 30 times; results are summarized in Table 9 and Fig. 10. One-way ANOVA confirmed significant differences among all seven algorithms for every layout–robot-count combination (all $p < 0.0001$). Because SGA-TS and SHC-GA frequently produced the lowest flowtime values, we performed a Wilcoxon signed-rank test on all 18 paired mean-flowtime data points (layouts (a)–(f) × 5, 10, 20 robots), yielding $p = 0.021$. Overall, SGA-TS achieved the lowest flowtime in 14 of 18 scenarios, with SHC-GA lowest in the remaining 4.

5.2.1. Layout complexity and dead-end effects

If there are no dead ends, robots can move efficiently from one waypoint to the next. However, when a robot encounters a dead end, it must enter the area, complete the task, and then exit—resulting in inefficient movement. The TaskShift algorithm incorporated in SGA-TS is designed to mitigate these inefficiencies by dynamically reallocating tasks away from problematic routes.

Fig. 11 visually demonstrates the improvement in flowtime due to TaskShift. Specifically, it highlights a scenario in layout (f) where the innermost shelves of the right warehouse are costly areas for robots to visit. Using SMP-GA, four robots visit these innermost shelves, which is detrimental to flowtime. The TaskShift algorithm reduces such cost waste by strategically reallocating tasks. The path lines in this figure should be interpreted as obstacle-avoiding geometric routes used to compute assignment costs. They do not represent a time-indexed multi-robot execution plan with robot-body clearance, reservation tables, or inter-robot collision avoidance; such synchronized execution planning is outside the offline allocation/sequencing scope of this study and is discussed as future work in the Conclusion.

Fig. 12 presents the per-layout flowtime improvement percentages of SGA-TS over each baseline. Each per-layout value was computed as the mean of scenario-wise improvements across the three robot counts (5, 10, and 20). The per-layout analysis reveals substantial variation. Layout (e) stands out as the scenario where SGA-TS achieves the largest flowtime improvements against every baseline: 44.2% over SMP-GA, 65.0% over IMC-GA, 33.9% over SHC-GA, 55.1% over GA-2OPT, 47.2% over ACO, and 53.3% over MO-MOEA. Layout (f) also shows strong improvements, particularly over SMP-GA (37.9%) and MO-MOEA (39.2%).

In contrast, layouts without dead ends — such as (a), (c), and (d) — show more moderate improvements ranging from 15.9% to 23.5% over SMP-GA and 18.8% to 24.7% over GA-2OPT. For SHC-GA, the flowtime improvements in these layouts are small (3.9%–5.6%), reflecting SHC-GA's competitive flowtime performance in simpler environments.

Notably, layout (b) also shows meaningful SHC-GA improvements (average 19.3% across 5, 10, and 20 robots), driven primarily by the 10-robot (27.5%) and 20-robot (30.1%) scenarios, while the 5-robot scenario shows a marginal 0.4% improvement.

These per-layout results suggest that the path-aware reassignment mechanism in SGA-TS is particularly beneficial in layouts containing dead ends, particularly layouts (e) and (f), where path-level inefficiencies are most prevalent.

5.2.2. Robot-to-waypoint ratio analysis

The improvement effect of SGA-TS varied depending on the ratio of robot count to waypoint count. For several baselines such as SMP-GA, SHC-GA, and GA-2OPT, larger improvements were generally observed at higher robot counts, whereas for ACO and MO-MOEA, the trend was not strictly monotonic. According to the experimental data (Table 9 and Fig. 13), for a robot count of 5, SGA-TS improved flowtime by 14.31% compared to SMP-GA, 46.83% compared to IMC-GA, 5.16% compared to SHC-GA, 19.18% compared to GA-2OPT, 32.94% compared to ACO, and 26.07% compared to MO-MOEA. For a robot count of 10, the improvements were 34.33% relative to SMP-GA, 51.61% relative to IMC-GA, 25.36% relative to SHC-GA, 38.57% relative to GA-2OPT, 28.13% relative to ACO, and 40.94% relative to MO-MOEA. When 20 robots were employed, SGA-TS showed improvements of 34.54% over SMP-GA, 46.20% over IMC-GA, 16.37% over SHC-GA, 41.42% over GA-2OPT, 24.04% over ACO, and 36.77% over MO-MOEA.

The relatively modest improvement observed with 5 robots can be attributed to the low robot-to-waypoint ratio. This results in clear clustering of the areas traversed by each robot, limiting the potential benefits of the TaskShift algorithm. In contrast, as the number of robots increases, the areas covered by the robots begin to overlap more, resulting in more frequent intervention by the TaskShift algorithm and, as a result, greater improvements in flowtime.

5.3. Verification with non-zero service time

In real-world applications such as warehouses, supermarkets, and airports, robots typically spend a certain amount of time at each task point to complete their assigned work (e.g., picking up or delivering items). To verify that SGA-TS remains effective under such conditions, we conducted additional experiments with a non-zero service time of 10 time units at each waypoint. The same six layouts (a)–(f) and three robot counts (5, 10, and 20) were used, with 30 repetitions per scenario. Tables 10 and 11 summarize the results.

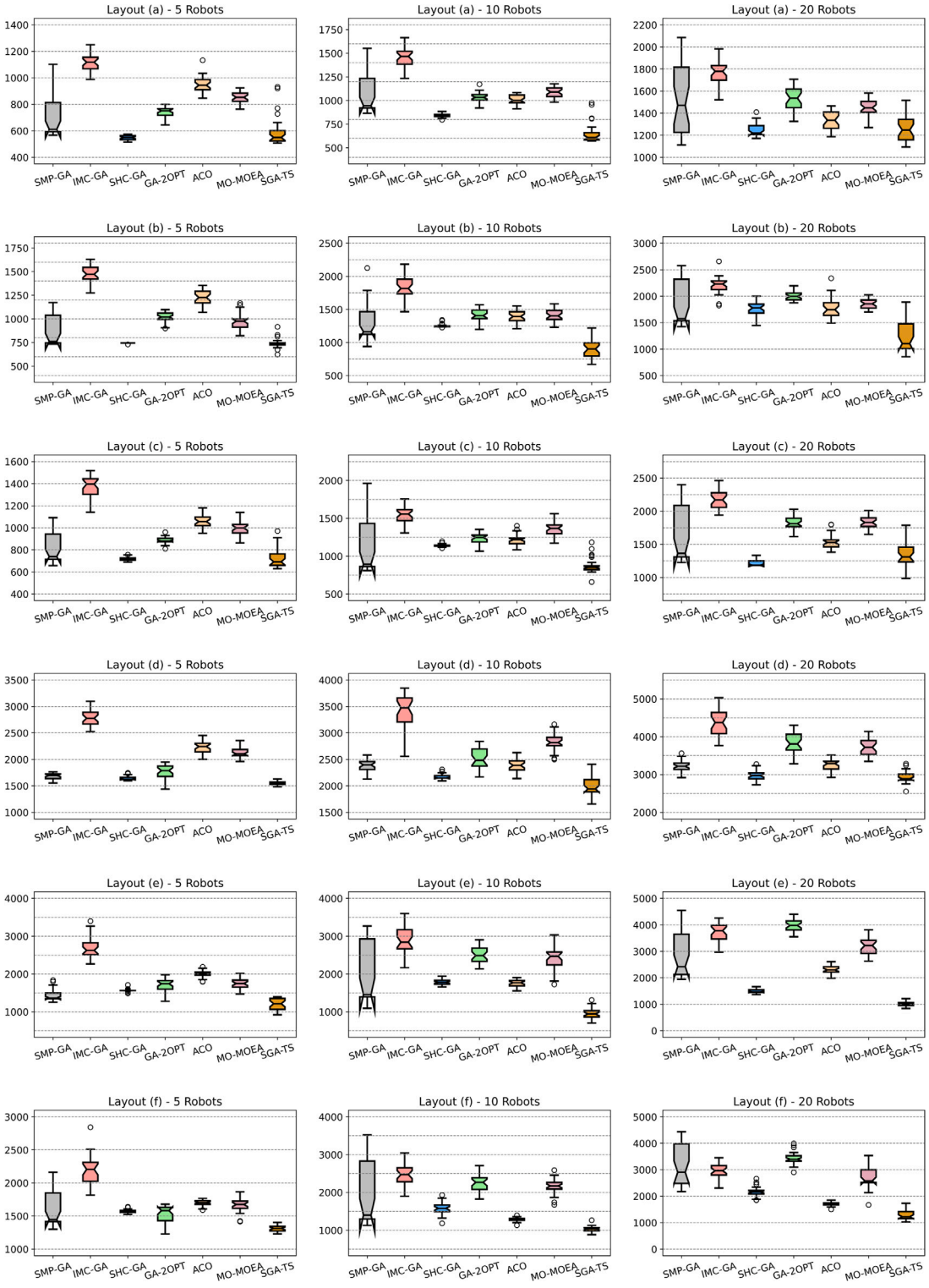


Fig. 10. Boxplot of flowtime. Boxplot conventions follow Fig. 7.

As shown in Tables 10 and 11, SGA-TS achieved the lowest makespan and flowtime in all 18 scenarios when a service time of 10 was applied. On average, SGA-TS improved the makespan by 6.32% over SMP-GA, 40.04% over IMC-GA, 16.14% over SHC-GA, 38.13% over GA-2OPT, 15.48% over ACO, and 15.42% over MO-MOEA. Similarly, the average flowtime was reduced by 6.47% compared to SMP-GA, 28.99% compared to IMC-GA, 9.59% compared to SHC-GA, 22.36% compared to GA-2OPT, 14.09% compared to ACO, and 18.86% compared to MO-MOEA. These results indicate that SGA-TS retains the lowest makespan and flowtime under non-zero service time conditions, suggesting that its performance advantage is not limited to the zero-service-time setting.

5.4. Runtime analysis

To address runtime scalability, we benchmarked all seven algorithms on the same 50 × 50 grid instance with 90 waypoints while

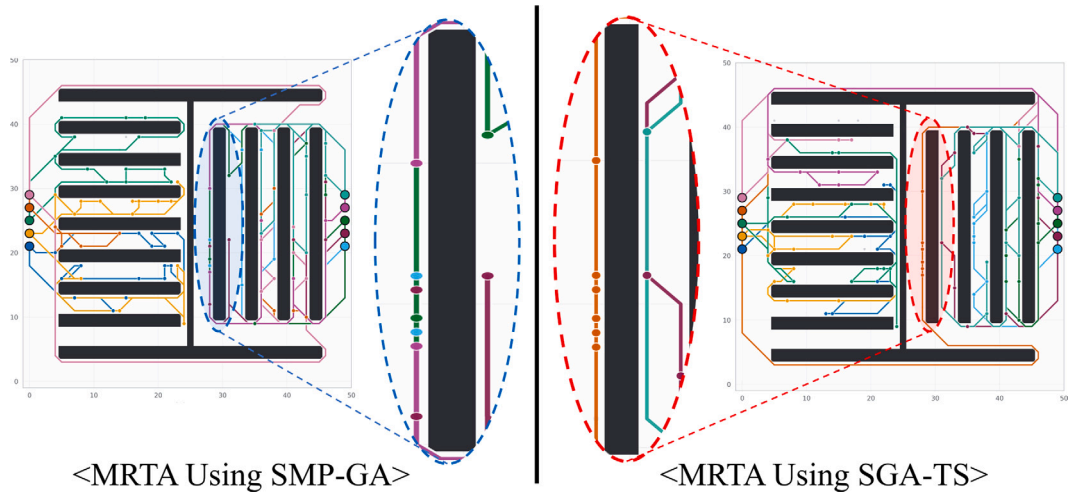


Fig. 11. Representative allocation and path-cost visualization comparing SGA-TS and SMP-GA. Colored paths denote independently computed obstacle-aware JPS traversal paths used for allocation-cost evaluation; they are not time-synchronized collision-free execution trajectories. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

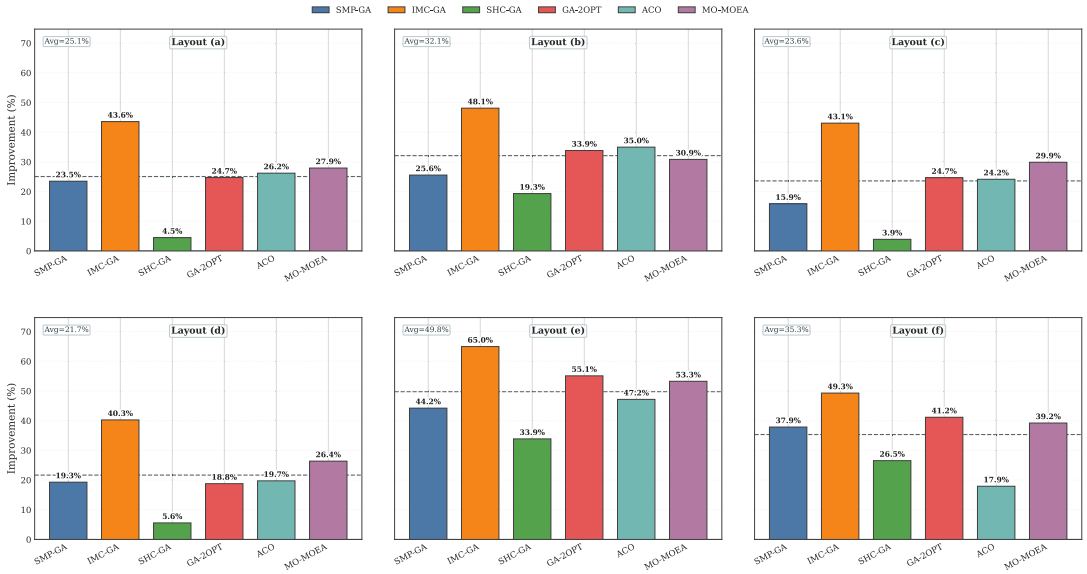


Fig. 12. Per-layout flowtime improvement (%) of SGA-TS over each baseline algorithm. Each bar represents the mean improvement across three robot counts (5, 10, and 20). Red bars indicate negative improvement (i.e., the baseline achieved lower flowtime than SGA-TS).

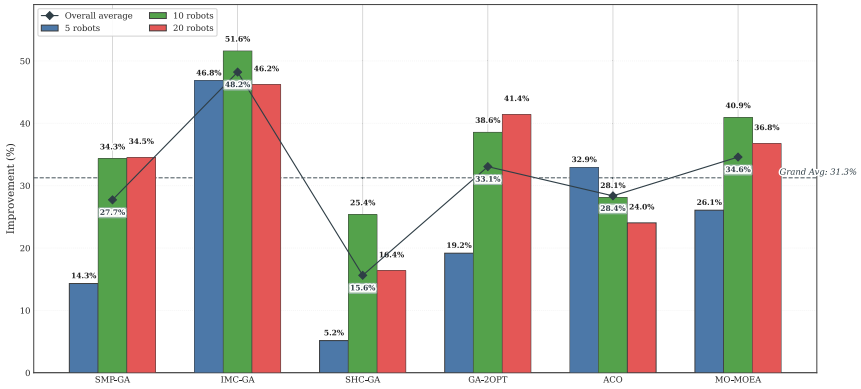


Fig. 13. Comparison of SGA-TS Flowtime Improvement (%) over Baseline Algorithms by Number of Robots.

varying the number of robots from 5 to 10 and 20. For fairness, scalar JPS distance-matrix construction was excluded from the timed portion and the same scalar cost information was made available to all algorithms; therefore, Table 12 reports only the optimization-loop

Table 10
Experiment result of makespan (Service Time = 10).

Layout	#Robots	SMP-GA		IMC-GA		SHC-GA		GA-2OPT		ACO		MO-MOEA		SGA-TS	
		Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev
(a)	5	298.50	2.80	444.40	21.00	298.00	3.70	407.30	22.10	379.50	15.20	353.10	7.90	**293.40**	2.60
	10	189.80	4.80	300.70	17.50	215.40	5.10	285.50	17.00	207.40	7.30	204.20	6.20	**178.30**	4.40
	20	130.00	1.70	190.60	10.60	132.60	4.80	198.00	13.70	137.00	6.70	137.10	3.90	**125.10**	5.20
(b)	5	337.00	1.30	514.90	23.60	356.00	0.00	471.40	22.20	436.60	15.80	385.20	10.10	**322.20**	3.50
	10	232.10	6.70	347.80	24.10	261.00	2.80	340.50	18.10	250.30	9.10	239.40	7.40	**220.40**	6.00
	20	158.90	5.00	219.60	12.60	148.70	6.60	237.20	13.20	167.40	8.40	147.20	4.40	**136.10**	5.70
(c)	5	351.90	4.30	510.30	25.50	367.40	5.60	458.30	22.20	417.70	13.80	402.20	11.20	**343.50**	3.00
	10	222.90	2.90	303.70	14.40	246.20	2.40	324.30	21.00	237.30	10.50	239.60	5.40	**213.90**	4.50
	20	135.20	3.40	218.10	11.40	137.70	5.10	226.70	11.30	155.70	6.80	149.50	4.90	**123.00**	7.40
(d)	5	521.70	9.60	780.90	31.80	556.30	15.10	667.20	35.90	647.50	22.20	610.80	13.80	**493.30**	9.50
	10	348.50	11.90	538.30	34.60	394.30	2.40	499.30	33.40	356.90	12.70	386.20	15.40	**308.90**	19.70
	20	237.30	11.50	360.70	17.60	252.30	17.90	393.50	15.00	247.60	10.90	250.60	11.70	**215.40**	9.80
(e)	5	474.10	11.90	806.90	65.60	541.40	5.30	623.10	30.00	591.90	18.40	552.00	25.30	**462.10**	17.20
	10	261.00	15.90	492.40	41.30	325.90	8.50	515.20	30.00	299.50	10.20	347.50	30.60	**251.50**	14.00
	20	220.80	22.90	335.10	25.10	344.40	20.60	438.20	27.30	217.70	7.60	236.30	14.30	**212.00**	27.10
(f)	5	457.40	28.90	684.00	38.90	503.90	9.40	605.30	30.20	538.80	14.80	510.30	21.70	**442.60**	11.60
	10	233.40	18.20	434.30	29.20	352.20	12.40	458.60	24.00	232.10	8.30	312.80	20.80	**207.50**	8.50
	20	208.20	28.30	434.30	29.20	244.90	9.40	363.90	16.00	198.80	15.10	202.10	17.60	**182.10**	17.90

Table 11
Experiment result of flowtime (Service Time = 10).

Layout	#Robots	SMP-GA		IMC-GA		SHC-GA		GA-2OPT		ACO		MO-MOEA		SGA-TS	
		Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev	Mean	Std.Dev
(a)	5	1463.23	11.93	2013.93	62.65	1454.37	4.56	1668.33	27.21	1861.97	71.87	1748.93	39.38	**1450.57**	13.96
	10	1800.30	48.19	2353.10	95.16	1932.43	16.66	2021.40	49.31	1953.00	56.38	1985.17	58.55	**1719.37**	41.02
	20	2238.93	100.04	2665.60	109.21	2161.92	70.32	2599.23	86.23	2332.83	81.97	2558.20	73.38	**2120.23**	55.11
(b)	5	1660.57	7.06	2369.60	94.87	1698.00	0.00	1924.07	57.18	2136.83	78.29	1901.43	51.25	**1597.20**	17.40
	10	2205.33	66.10	2741.73	157.48	2347.57	24.96	2409.33	67.81	2357.50	81.41	2329.17	71.34	**2115.53**	46.92
	20	2779.00	85.65	3103.10	157.82	2417.89	88.12	3134.07	75.64	2771.37	108.77	2745.17	102.90	**2372.37**	76.91
(c)	5	1735.63	19.02	2350.90	99.73	1757.33	23.71	1866.63	39.52	2057.23	62.90	1992.63	54.45	**1700.13**	13.33
	10	2154.03	30.05	2527.43	103.06	2317.33	24.66	2315.60	63.65	2229.80	89.92	2336.73	56.60	**2074.17**	30.16
	20	2325.87	74.38	3149.40	144.57	2274.41	44.14	3010.13	71.48	2655.53	83.58	2810.57	101.92	**2232.80**	88.76
(d)	5	2567.30	50.00	3687.30	141.59	2580.87	37.11	2786.20	53.08	3173.50	104.06	3012.37	75.39	**2446.53**	48.19
	10	3250.20	125.50	4312.60	301.90	3248.00	28.39	3742.27	102.69	3365.03	117.02	3733.13	148.82	**2945.87**	139.92
	20	4104.90	223.95	5279.33	337.29	3904.06	167.96	5108.53	198.64	4166.03	162.82	4658.90	252.46	**3828.73**	119.09
(e)	5	2314.10	66.80	3601.80	267.99	2527.33	23.14	2730.80	85.62	2904.47	93.61	2717.77	129.83	**2295.27**	87.87
	10	2411.08	139.56	3772.13	347.66	2638.50	102.19	3940.67	110.37	2745.60	98.09	3338.77	301.05	**2365.13**	110.12
	20	3588.77	352.26	4626.47	326.31	5194.63	379.65	5304.17	205.03	3358.43	163.45	4226.33	282.15	**2968.23**	563.98
(f)	5	2250.10	144.99	3094.93	208.82	2382.77	26.32	2531.57	56.76	2659.90	73.52	2525.97	109.12	**2190.63**	68.77
	10	2217.33	168.14	3374.13	291.94	2664.40	136.41	3450.93	99.93	2191.83	70.94	3037.40	201.45	**1970.53**	74.50
	20	3348.10	391.45	3374.13	291.94	3123.43	282.39	4713.33	196.00	2785.67	106.75	3671.07	324.37	**2712.27**	345.33

Table 12
Runtime comparison of all algorithms under larger robot-team sizes (50 × 50 grid, 90 waypoints, mean ± std over 30 independent trials). Distance-matrix precomputation is excluded; only the optimization loop is timed.

Algorithm	5 robots	10 robots	20 robots	20/5 growth
SGA-TS	2.81 ± 0.06	3.38 ± 0.04	4.48 ± 0.12	1.59×
ACO	44.37 ± 0.51	40.95 ± 0.22	39.53 ± 0.79	0.89×
SMP-GA	2.15 ± 0.03	2.49 ± 0.02	3.24 ± 0.08	1.51×
MO-MOEA	4.36 ± 0.06	4.87 ± 0.02	6.26 ± 0.34	1.44×
IMC-GA	1.86 ± 0.06	2.32 ± 0.05	3.46 ± 0.22	1.86×
GA-2OPT	4.72 ± 0.02	4.97 ± 0.03	6.05 ± 0.15	1.28×
SHC-GA	1.77 ± 0.02	1.99 ± 0.03	2.55 ± 0.07	1.44×

runtime. Each algorithm was executed for 30 independent trials using the same hardware and the same algorithmic settings as in the main experiments.

The runtime of SGA-TS increases from 2.81 s with 5 robots to 4.48 s with 20 robots, corresponding to a 1.59× increase under a fourfold increase in the number of robots. This trend is consistent with the complexity analysis in Section 4.3: the waypoint count dominates the problem size in this instance, while the additional robot-dependent terms grow moderately. SGA-TS is slower than the simplest GA variants because TaskShift performs path-aware reassignment, but its runtime remains within the same order of magnitude as the GA-based baselines and remains substantially faster than ACO across all tested robot counts.

5.5. Performance result discussion

As reported in Sections 5.1–5.2, SGA-TS achieved the lowest mean makespan in all 18 scenarios, with average improvements of 17%–48% over the six baselines, and the lowest flowtime in 14 of 18 scenarios,

Table 13
Average improvement (%) of SGA-TS over baseline algorithms by robot count. Both makespan and flowtime improvements are averaged over all six baselines (SMP-GA, IMC-GA, SHC-GA, GA-2OPT, ACO, MO-MOEA) across six layouts.

n	W/n	$\rho \approx n/W$	Makespan Imp. (%)	Flowtime Imp. (%)
5	∼18	0.056	28.8	24.1
10	∼9	0.111	35.9	36.5
20	∼4.5	0.222	28.4	33.2

with average improvements of 10%–48%. Although SGA-TS was primarily designed to minimize makespan for deadline-critical scenarios, the TaskShift operator also reduced flowtime by eliminating path-level inefficiencies such as unnecessary detours and redundant waypoint visits. Furthermore, the additional experiments with a non-zero service time (Section 5.3) showed that SGA-TS achieved the lowest makespan and flowtime in all 18 scenarios under this setting, indicating that its advantage is not specific to the zero-service-time case.

To further analyze the effect of the robot-to-task ratio on SGA-TS performance, we define $\rho = n/W$, where n is the number of robots and W is the number of waypoints. Since W varies slightly across layouts ($W = 90$ for layouts (a), (b), (d)–(f) and $W = 98$ for layout (c)), the reported ρ values represent approximate ratios. Table 13 summarizes the average improvements of SGA-TS grouped by robot count, with both columns averaging over all six baselines across six layouts.

Both makespan and flowtime improvements peak at $n = 10$ ($\rho \approx 0.11$), where each robot manages approximately 9 waypoints. At the lowest ratio ($n = 5$, $\rho \approx 0.06$), each robot covers a spatially distinct cluster of ∼18 waypoints, resulting in limited route overlap among robots and fewer opportunities for the TaskShift operator to transfer waypoints across routes. At the highest tested ratio ($n = 20$, $\rho \approx 0.22$), each robot visits only ∼4.5 waypoints on average; while route overlap is more frequent, the reduced per-robot task count limits the marginal benefit of further redistribution. Notably, the trend is not strictly monotonic across individual baselines — for instance, improvements over ACO and MO-MOEA show non-monotonic patterns — indicating that the optimal ratio depends on both the baseline algorithm characteristics and the layout structure. These observations suggest that SGA-TS is most effective at moderate robot-to-task ratios where each robot carries enough tasks for meaningful route optimization and sufficient route overlap exists for TaskShift to operate. The contrasting behavior of the two metrics at $n = 20$ can be explained by their aggregation semantics. Makespan, defined as $\max_k \mathrm{Cost}_k(x)$, improves only when the bottleneck robot's cost decreases. At high robot counts, the bottleneck carries few waypoints and its cost approaches a geographic lower bound, limiting the marginal benefit of further transfers under the bottleneck-first rule. Flowtime, defined as $\sum_k \mathrm{Cost}_k(x)$, benefits from any per-robot saving: when TaskShift absorbs a waypoint that already lies on the receiver's path, the receiver incurs near-zero additional cost while the donor eliminates a detour. With 20 robots forming a dense route network, such efficient absorptions remain frequent, and their cumulative savings sustain the flowtime improvement even as makespan gains diminish.

6. Conclusion

This study proposes SGA-TS, which combines a Seeding Genetic Algorithm with the TaskShift operator to optimize robot task assignment for MRTA in a grid-based ST–SR–TA setting. The objective function targets makespan minimization, while the TaskShift operator — integrated into the mutation stage — reduces flowtime by reassigning tasks to eliminate path-level inefficiencies. Across 18 experimental scenarios (six layouts, three robot counts) and six baseline algorithms, SGA-TS achieved the lowest makespan in all 18 cases and the lowest flowtime in 14 of 18 cases. Additional experiments with non-zero service times confirmed these findings: SGA-TS achieved the lowest makespan and flowtime in all 18 scenarios. The additional runtime benchmark shows that SGA-TS scales moderately from 2.81 s with 5 robots to 4.48 s with 20 robots on the tested instance, while remaining substantially faster than ACO.

Nevertheless, the current study has several limitations that suggest directions for future research. First, SGA-TS is a centralized, offline algorithm that assumes all task locations are known prior to execution; it does not support dynamic task arrivals or online re-allocation during mission execution. Second, the current formulation does not account for inter-robot collision avoidance, which would require integration with Multi-Agent Path Finding (MAPF) methods such as Conflict-Based Search (CBS) [66] or Priority Inheritance with Backtracking (PIBT) [67]. Third, the makespan-first objective function limits flowtime competitiveness in a subset of scenarios; incorporating an adaptive weighting scheme or a constrained bi-objective formulation could improve flowtime performance without materially degrading makespan performance. Addressing these limitations constitutes our future research agenda.

CRediT authorship contribution statement

JangHo Seo: Writing – original draft, Visualization, Validation, Software, Resources, Investigation, Data curation. **Joonwoo Lee:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. NRF-2020R1C1C1008520, RS-2026-25491593).

Data availability

Data will be made available on request.

References

[1] J. Melvin, P. Keskinocak, S. Koenig, C. Tovey, B.Y. Ozkaya, Multi-robot routing with rewards and disjoint time windows, in: 2007 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, 2007, pp. 2332–2337.
[2] K.M. Wurm, C. Stachniss, W. Burgard, Coordinated multi-robot exploration using a segmentation of the environment, in: 2008 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, 2008, pp. 1160–1165.
[3] C. Ju, J. Kim, J. Seol, H.I. Son, A review on multirobot systems in agriculture, Comput. Electron. Agric. 202 (2022) 107336.
[4] J. Gregory, J. Fink, E. Stump, J. Twigg, J. Rogers, D. Baran, N. Fung, S. Young, Application of multi-robot systems to disaster-relief scenarios with limited communication, in: Field and Service Robotics: Results of the 10th International Conference, Springer, 2016, pp. 639–653.
[5] H. Kitano, S. Tadokoro, I. Noda, H. Matsubara, T. Takahashi, A. Shinjou, S. Shimada, Robocup rescue: Search and rescue in large-scale disasters as a domain for autonomous agents research, in: IEEE SMC'99 Conference Proceedings. 1999 IEEE International Conference on Systems, Man, and Cybernetics (Cat. No. 99CH37028), Vol. 6, IEEE, 1999, pp. 739–743.
[6] C. Liu, A. Kroll, A centralized multi-robot task allocation for industrial plant inspection by using a* and genetic algorithms, in: Artificial Intelligence and Soft Computing: 11th International Conference, ICAISC 2012, Zakopane, Poland, April 29-May 3, 2012, Proceedings, Part II 11, Springer, 2012, pp. 466–474.
[7] C. Liu, A. Kroll, Memetic algorithms for optimal task allocation in multi-robot systems for inspection problems with cooperative tasks, Soft Comput. 19 (2015) 567–584.

[8] N. Correll, A. Martinoli, Multirobot inspection of industrial machinery, IEEE Robot. Autom. Mag. 16 (1) (2009) 103–112.
[9] H. Afzaal, N.A. Zafar, Robot-based forest fire detection and extinguishing model, in: 2016 2nd International Conference on Robotics and Artificial Intelligence, ICRAI, IEEE, 2016, pp. 112–117.
[10] S. Patnaik, L.C. Jain, S.G. Tzafestas, G. Resconi, A. Konar (Eds.), Innovations in Robot Mobility and Control, in: Studies in Computational Intelligence, 8, Springer Berlin Heidelberg, 2005, http://dx.doi.org/10.1007/b96087.
[11] D. Portugal, R. Rocha, A survey on multi-robot patrolling algorithms, in: Technological Innovation for Sustainability: Second IFIP WG 5.5/SOCOLNET Doctoral Conference on Computing, Electrical and Industrial Systems, DoCEIS 2011, Costa de Caparica, Portugal, February 21-23, 2011. Proceedings 2, Springer, 2011, pp. 139–146.
[12] B. Mishra, D. Garg, P. Narang, V. Mishra, Drone-surveillance for search and rescue in natural disaster, Comput. Commun. 156 (2020) 1–10.
[13] K. Nagatani, Y. Okada, N. Tokunaga, S. Kiribayashi, K. Yoshida, K. Ohno, E. Takeuchi, S. Tadokoro, H. Akiyama, I. Noda, et al., Multirobot exploration for search and rescue missions: A report on map building in RoboCupRescue 2009, J. Field Robot. 28 (3) (2011) 373–387.
[14] G.P. Das, T.M. McGinnity, S.A. Coleman, L. Behera, A distributed task allocation algorithm for a multi-robot system in healthcare facilities, J. Intell. Robot. Syst. 80 (2015) 33–58.
[15] J. Sustarevas, K.B. Tan, D. Gerber, R. Stuart-Smith, V.M. Pawar, YouWasps: Towards autonomous multi-robot mobile deposition for construction, in: 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, IEEE, 2019, pp. 2320–2327.
[16] K. Dörfler, G. Dielemans, L. Lachmayer, T. Recker, A. Raatz, D. Lowke, M. Gerke, Additive manufacturing using mobile robots: Opportunities and challenges for building construction, Cem. Concr. Res. 158 (2022) 106772.
[17] M. Shelkamy, C.M. Elias, D.M. Mahfouz, O.M. Shehata, Comparative analysis of various optimization techniques for solving multi-robot task allocation problem, in: 2020 2nd Novel Intelligent and Leading Emerging Sciences Conference, NILES, IEEE, 2020, pp. 538–543.
[18] X. Zhou, H. Wang, B. Ding, T. Hu, S. Shang, Balanced connected task allocations for multi-robot systems: An exact flow-based integer program and an approximate tree-based genetic algorithm, Expert Syst. Appl. 116 (2019) 10–20.
[19] J.G. Martin, J.R.D. Frejo, R.A. García, E.F. Camacho, Multi-robot task allocation problem with multiple nonlinear criteria using branch and bound and genetic algorithms, Intell. Serv. Robot. 14 (5) (2021) 707–727.
[20] Y. Tatsumoto, M. Shiraishi, K. Cai, Z. Lin, Application of online supervisory control of discrete-event systems to multi-robot warehouse automation, Control Eng. Pract. 81 (2018) 97–104.
[21] A. Ham, M.-J. Park, Human–robot task allocation and scheduling: Boeing 777 case study, IEEE Robot. Autom. Lett. 6 (2) (2021) 1256–1263.
[22] W. Liu, Z. Kuang, Y. Zhang, B. Zhou, P. He, S. Li, An effective hybrid genetic algorithm for the multi-robot task allocation problem with limited span, Expert Syst. Appl. 280 (2025) 127299, http://dx.doi.org/10.1016/j.eswa.2025.127299.
[23] J. Chen, J. Wang, Q. Xiao, C. Chen, A multi-robot task allocation method based on multi-objective optimization, in: 2018 15th International Conference on Control, Automation, Robotics and Vision, ICARCV, IEEE, 2018, pp. 1868–1873.
[24] S. Yang, Y. Zhang, L. Ma, Y. Song, P. Zhou, G. Shi, H. Chen, A novel maximin-based multi-objective evolutionary algorithm using one-by-one update scheme for multi-robot scheduling optimization, IEEE Access 9 (2021) 121316–121328.
[25] S. Bandi, D. Thalmann, Space discretization for efficient human navigation, in: Computer Graphics Forum, Vol. 17, Wiley Online Library, 1998, pp. 195–206.
[26] J. Lee, Heterogeneous-ants-based path planner for global path planning of mobile robot applications, Int. J. Control. Autom. Syst. 15 (2017) 1754–1769.
[27] L. Liang, L. Zhu, W. Jia, X. Cheng, Multirobot task planning method based on the energy penalty strategy, Appl. Sci. 13 (8) (2023) 4887.
[28] A.A.H. Qizilbash, C. Henkel, S. Mostaghim, Ant colony optimization based multi-robot planner for combined task allocation and path finding, in: 2020 17th International Conference on Ubiquitous Robots, UR, IEEE, 2020, pp. 487–493.
[29] T. Ma, H. Zhao, X. Li, F. Yang, C.S. Liu, J. Liu, A coarse-and fine-grained niching-based differential evolution for multimodal optimization problems and its application in multirobot task allocation, Swarm Evol. Comput. 83 (2023) 101412.
[30] J. Xin, Q. Yuan, A. D'Ariano, G. Guo, Y. Liu, Y. Zhou, Dynamic unbalanced task allocation of warehouse AGVs using integrated adaptive large neighborhood search and Kuhn–Munkres algorithm, Comput. Ind. Eng. 195 (2024) 110410.
[31] B.P. Gerkey, M.J. Matarić, A formal analysis and taxonomy of task allocation in multi-robot systems, Int. J. Robot. Res. 23 (9) (2004) 939–954.
[32] D. Harabor, A. Grastien, Online graph pruning for pathfinding on grid maps, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 25, 2011, pp. 1114–1119.
[33] Z. Lu, K. Zhang, J. He, Y. Niu, Applying k-means clustering and genetic algorithm for solving mtsp, in: Bio-Inspired Computing–Theories and Applications: 11th International Conference, BIC-TA 2016, Xi'an, China, October 28-30, 2016, Revised Selected Papers, Part II 11, Springer, 2016, pp. 278–284.
[34] R. Necula, M. Breaban, M. Raschip, Tackling the bi-criteria facet of multiple traveling salesman problem with ant colony systems, in: 2015 IEEE 27th International Conference on Tools with Artificial Intelligence, ICTAI, IEEE, 2015, pp. 873–880.
[35] S.S. Choong, L.-P. Wong, C.P. Lim, An artificial bee colony algorithm with a modified choice function for the traveling salesman problem, Swarm Evol. Comput. 44 (2019) 622–635.
[36] C. Liu, Y. Zhang, Research on MTSP problem based on simulated annealing, in: Proceedings of the 1st International Conference on Information Science and Systems, 2018, pp. 283–285.
[37] H. Liang, Y. Ma, Z. Cao, T. Liu, F. Ni, Z. Li, J. Hao, SplitNet: A reinforcement learning based sequence splitting method for the MinMax multiple travelling salesman problem, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 8720–8727.
[38] J. Park, C. Kwon, J. Park, Learn to solve the min-max multiple traveling salesmen problem with reinforcement learning, in: Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems, 2023, pp. 878–886.
[39] J.H. Holland, Genetic algorithms, Sci. Am. 267 (1) (1992) 66–73.
[40] M. Sedighpour, M. Yousefikhoshbakht, N. Mahmoodi Darani, An effective genetic algorithm for solving the multiple traveling salesman problem, J. Optim. Ind. Eng. 4 (8) (2011) 73–79.
[41] C. Jianping, Y. Yimin, W. Yunbiao, Multi-robot task allocation based on robotic utility value and genetic algorithm, in: 2009 IEEE International Conference on Intelligent Computing and Intelligent Systems, Vol. 2, IEEE, 2009, pp. 256–260.
[42] A. Khamis, A. Hussein, A. Elmogy, Multi-robot task allocation: A review of the state-of-the-art, in: Cooperative Robots and Sensor Networks 2015, Springer, 2015, pp. 31–51.
[43] H. Chakraa, F. Guerin, E. Leclercq, D. Lefebvre, Optimization techniques for multi-robot task allocation problems: Review on the state-of-the-art, Robot. Auton. Syst. 168 (2023) 104492.
[44] K.A. Athira, J. Divya Udayan, U. Subramaniam, A systematic literature review on multi-robot task allocation, ACM Comput. Surv. 57 (3) (2024) 1–28.
[45] E. Lam, D.D. Harabor, P.J. Stuckey, J. Li, Exact anytime multi-agent path finding using branch-and-cut-and-price and large neighborhood search, in: Proceedings of the International Conference on Automated Planning and Scheduling, Vol. 33, 2023, pp. 254–258.
[46] T. Zheng, L. Yang, Optimal ant colony algorithm based multi-robot task allocation and processing sequence scheduling, in: 2008 7th World Congress on Intelligent Control and Automation, IEEE, 2008, pp. 5693–5698, http://dx.doi.org/10.1109/WCICA.2008.4593859.
[47] F. Janati, F. Abdollahi, S.S. Ghidary, M. Jannatifar, J. Baltes, S. Sadeghnejad, Multi-robot task allocation using clustering method, in: Robot Intelligence Technology and Applications 4: Results from the 4th International Conference on Robot Intelligence Technology and Applications, Springer, 2017, pp. 233–247.
[48] X. Shan, Y. Jin, M. Jurt, P. Li, A distributed multi-robot task allocation method for time-constrained dynamic collective transport, Robot. Auton. Syst. 178 (2024) 104722, http://dx.doi.org/10.1016/j.robot.2024.104722.
[49] M.J. Bagchi, S.B. Nair, P.K. Das, On a dynamic and decentralized energy-aware technique for multi-robot task allocation, Robot. Auton. Syst. 180 (2024) 104762, http://dx.doi.org/10.1016/j.robot.2024.104762.
[50] S. Paul, P. Ghassemi, S. Chowdhury, Learning scalable policies over graphs for multi-robot task allocation using capsule attention networks, in: 2022 International Conference on Robotics and Automation, ICRA, IEEE, 2022, pp. 8815–8822, http://dx.doi.org/10.1109/ICRA46639.2022.9812370.
[51] A. Király, J. Abonyi, A novel approach to solve multiple traveling salesmen problem by genetic algorithm, in: I.J. Rudas, J. Fodor, J. Kacprzyk (Eds.), Computational Intelligence in Engineering, in: Studies in Computational Intelligence, 313, Springer Berlin Heidelberg, 2010, pp. 141–151, http://dx.doi.org/10.1007/978-3-642-15220-7_12.
[52] K. Jose, D.K. Pratihar, Task allocation and collision-free path planning of centralized multi-robots system for industrial plant inspection using heuristic methods, Robot. Auton. Syst. 80 (2016) 34–42.
[53] C.-H. Wang, Q.-K. Pan, X.-P. Li, H.-Y. Sang, B. Wang, A multi-objective teaching-learning-based optimizer for a cooperative task allocation problem of weeding robots and spraying drones, Swarm Evol. Comput. 87 (2024) 101565, http://dx.doi.org/10.1016/j.swevo.2024.101565.
[54] S. Liu, B. Feng, Y. Bi, D. Yu, An integrated approach to precedence-constrained multi-agent task assignment and path finding for mobile robots in smart manufacturing, Appl. Sci. 14 (7) (2024) 3094, http://dx.doi.org/10.3390/app14073094.
[55] Z. Qiu, J. Long, Y. Yu, S. Chen, Integrated task assignment and path planning for multi-type robots in an intelligent warehouse system, Transp. Res. E 194 (2025) 103883, http://dx.doi.org/10.1016/j.tre.2024.103883.
[56] J. David, R. Valencia, Simultaneous path planning and task allocation in dynamic environments, Robotics 14 (2) (2025) 17, http://dx.doi.org/10.3390/robotics14020017.
[57] H. Chakraa, E. Leclercq, F. Guérin, D. Lefebvre, Integrating collision avoidance strategies into multi-robot task allocation for inspection, Trans. Inst. Meas. Control 47 (7) (2025) 1466–1477, http://dx.doi.org/10.1177/01423312241295828.

[58] L. Yang, P. Li, T. Wang, J. Miao, J. Tian, C. Chen, J. Tan, Z. Wang, Multi-area collision-free path planning and efficient task scheduling optimization for autonomous agricultural robots, Sci. Rep. 14 (2024) 18347, http://dx.doi.org/10.1038/s41598-024-69265-y.

[59] S. Lee, J. Sim, C. Nam, Very large-scale multi-robot task allocation in challenging environments via robot redistribution, Robot. Auton. Syst. 194 (2025) 105126, http://dx.doi.org/10.1016/j.robot.2025.105126.

[60] O. Bräysy, M. Gendreau, Vehicle routing problem with time windows, Part I: Route construction and local search algorithms, Transp. Sci. 39 (1) (2005) 104–118, http://dx.doi.org/10.1287/trsc.1030.0056.

[61] T. Vidal, T.G. Crainic, M. Gendreau, C. Prins, A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows, Comput. Oper. Res. 40 (1) (2013) 475–489, http://dx.doi.org/10.1016/j.cor.2012.07.018.

[62] I. Kara, T. Bektas, Integer linear programming formulations of multiple salesman problems and its variations, European J. Oper. Res. 174 (3) (2006) 1449–1458.

[63] P. Moscato, On Evolution, Search, Optimization, Genetic Algorithms and Martial Arts: Towards Memetic Algorithms, Tech. Rep. C3P Report 826, Caltech Concurrent Computation Program, 1989.

[64] F. Neri, C. Cotta, Memetic algorithms and memetic computing optimization: A literature review, Swarm Evol. Comput. 2 (2012) 1–14, http://dx.doi.org/10.1016/j.swevo.2011.11.003.

[65] S.-H. Chan, R. Stern, A. Felner, S. Koenig, Greedy priority-based search for sub-optimal multi-agent path finding, in: Proceedings of the International Symposium on Combinatorial Search, Vol. 16, 2023, pp. 11–19.

[66] G. Sharon, R. Stern, A. Felner, N.R. Sturtevant, Conflict-based search for optimal multi-agent pathfinding, Artificial Intelligence 219 (2015) 40–66.

[67] K. Okumura, M. Machida, X. Défago, Y. Tamura, Priority inheritance with backtracking for iterative multi-agent path finding, Artificial Intelligence 310 (2022) 103752.